

1. Классы и объекты. Абстракция данных.

Классы и объекты

Класс — это пользовательский тип данных, представляющий собой шаблон для создания объектов. Класс описывает структуру и поведение объектов с помощью:

- полей (переменных экземпляра), определяющих состояние объекта;
- методов, определяющих его поведение.

Пример объявления класса в Java:

```
class Person {  
    String name;  
    int age;  
  
    void sayHello() {  
        System.out.println("Hello");  
    }  
}
```

Объект — это конкретный экземпляр класса, создаваемый в памяти во время выполнения программы. Объекты обладают собственным состоянием и используют методы, описанные в классе.

Пример создания объекта:

```
Person p = new Person();  
p.name = "Ivan";  
p.age = 20;
```

Таким образом, класс задаёт описание, а объект является его реализацией.

Абстракция данных

Абстракция данных — это принцип ООП, заключающийся в отделении интерфейса объекта от его реализации. Пользователю предоставляются только необходимые способы взаимодействия с объектом, при этом детали внутренней реализации скрыты.

Цель абстракции:

- уменьшение сложности программ;
- повышение надёжности и безопасности кода;
- упрощение сопровождения и расширения системы.

Реализация абстракции данных в Java

В языке Java абстракция данных достигается следующими механизмами:

1. Инкапсуляция

Реализуется с помощью модификаторов доступа (`private`, `public`, `protected`) и методов доступа (`getter` и `setter`). Внутренние данные скрываются от прямого доступа извне.

2. Абстрактные классы

Абстрактный класс объявляется с помощью ключевого слова `abstract` и может содержать абстрактные методы, не имеющие реализации. Экземпляры абстрактных классов создавать нельзя.

3. Интерфейсы

Интерфейс определяет набор методов, которые должен реализовать класс. Он задаёт поведение, не раскрывая деталей реализации, тем самым обеспечивая высокий уровень абстракции.

2. Переменные класса и константы. Ограничение доступа.

Переменные класса

Переменные класса — это переменные, объявленные с модификатором `static`.

Они принадлежат **классу в целом**, а не отдельному объекту, и существуют в единственном экземпляре независимо от количества созданных объектов.

Характерные особенности:

- создаются один раз при загрузке класса;
- общие для всех объектов данного класса;
- обращение возможно без создания объекта.

Пример:

```
class Counter {
    static int count = 0;

    Counter() {
        count++;
    }
}
```

В данном примере переменная `count` хранит общее количество созданных объектов класса `Counter`.

Константы

Константа — это переменная, значение которой не может быть изменено после инициализации.

В Java константы объявляются с помощью ключевого слова `final`.

Чаще всего константы объявляются как:

```
public static final double PI = 3.14159;
```

Особенности констант:

- значение присваивается один раз;
- не может быть изменено в ходе выполнения программы;
- обычно используются для хранения фиксированных значений (настроек, математических величин);
- по соглашению именуются заглавными буквами.

Ограничение доступа (модификаторы доступа)

Ограничение доступа — это механизм, позволяющий управлять видимостью классов, полей и методов.

В Java используются следующие модификаторы доступа:

1. **public**

Доступ разрешён из любого места программы.

2. **protected**

Доступ разрешён:

- внутри того же пакета;
- в классах-наследниках, даже если они находятся в другом пакете.

3. **(package-private)** — доступ по умолчанию

Если модификатор не указан, доступ разрешён только внутри одного пакета.

4. **private**

Доступ разрешён только внутри данного класса.

Пример:

```
class Person {
    private String name;
    public int age;

    private void setName(String name) {
        this.name = name;
    }
}
```

Назначение ограничения доступа

Использование модификаторов доступа позволяет:

- защитить внутренние данные класса от некорректного использования;
- реализовать принцип инкапсуляции;
- снизить связанность между компонентами программы;
- упростить сопровождение и модификацию кода.

3. Модификаторы уровня доступа.

Модификаторы уровня доступа в Java

Модификаторы уровня доступа в языке Java используются для управления видимостью классов, полей, методов и конструкторов. Они позволяют реализовать инкапсуляцию и защитить внутреннюю реализацию программных компонентов от некорректного использования.

Виды модификаторов доступа

В Java предусмотрено **четыре уровня доступа**:

1. **public**

Обеспечивает максимальный уровень доступа.

Элемент, объявленный как `public`, доступен из любого класса и любого пакета.

2. **protected**

Доступ разрешён:

- внутри того же пакета;
- в классах-наследниках, даже если они находятся в другом пакете.

3. **Доступ по умолчанию (package-private)**

Если модификатор не указан, элемент доступен только внутри своего пакета.

Используется для логической группировки классов внутри одного пакета.

4. **private**

Обеспечивает минимальный уровень доступа.

Элемент доступен только внутри класса, в котором он объявлен.

Применение модификаторов доступа

Модификаторы доступа могут применяться к:

- классам;
- полям;
- методам;
- конструкторам.

Следует учитывать, что:

- классы верхнего уровня могут быть только `public` или `package-private`;
- поля и методы могут иметь любой из четырёх уровней доступа.

Значение модификаторов доступа

Использование модификаторов доступа позволяет:

- реализовать принцип инкапсуляции;
- повысить безопасность программ;
- уменьшить зависимость между классами;
- упростить сопровождение и тестирование кода.

4. Специальные методы классов (конструкторы). Конструктор по умолчанию. Конструкторы при наследовании в Java.

Специальные методы классов (конструкторы) в Java

Понятие конструктора

Конструктор — это специальный метод класса, предназначенный для: 1)создания объекта; 2)инициализации его полей.

Особенности конструктора: 1)имя конструктора совпадает с именем класса; 2)не имеет возвращаемого типа (даже `void`); 3)вызывается автоматически при создании объекта с помощью оператора `new`.

Пример:

```
class Person {
    String name;
    int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Конструктор по умолчанию

Конструктор по умолчанию — это конструктор без параметров.

Особенности:

- если в классе **не объявлен ни один конструктор**, компилятор автоматически создаёт конструктор по умолчанию;
- если в классе объявлен **хотя бы один конструктор**, автоматический конструктор по умолчанию **не создаётся**.

Пример:

```
class A {
    int x;
}
```

Компилятор неявно создаёт:

```
A() {
}
```

Перегрузка конструкторов

В одном классе может быть несколько конструкторов с разными параметрами — это называется **перегрузкой конструкторов**.

Пример:

```
class Point {
    int x, y;

    Point() {
        x = 0;
        y = 0;
    }

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

```
}  
}
```

Конструкторы при наследовании в Java

Вызов конструктора базового класса

При создании объекта подкласса:

- **сначала вызывается конструктор суперкласса;**
- затем — конструктор подкласса.

Если вызов конструктора суперкласса не указан явно, компилятор автоматически вставляет:

```
super();
```

Пример:

```
class A {  
    A() {  
        System.out.println("A");  
    }  
}
```

```
class B extends A {  
    B() {  
        System.out.println("B");  
    }  
}
```

Результат создания `new B()`:

A
B

Явный вызов конструктора суперкласса

Если в суперклассе нет конструктора без параметров, подкласс **обязан** явно вызвать нужный конструктор с помощью `super(...)`.

Пример:

```
class A {  
    A(int x) { }  
}  
  
class B extends A {  
    B(int x) {  
        super(x);  
    }  
}
```

! Вызов `super(...)`:

- должен быть **первой строкой** конструктора;
- может использоваться только один раз.

Основные правила работы конструкторов при наследовании

- Конструкторы **не наследуются**;
- Каждый класс отвечает за инициализацию своих полей;
- Без корректного вызова конструктора суперкласса объект не будет создан.

5. Класс Object. Основные методы класса Object

Назначение класса Object

Object — это базовый класс в языке Java.

Все классы в Java **неявно наследуются от класса Object**, если явно не указан другой суперкласс.

Таким образом, Object является корнем иерархии классов.

Это означает, что любой объект в Java:

- может быть приведён к типу Object;
- наследует методы, определённые в классе Object.

Основные методы класса Object

Класс Object определяет набор базовых методов, обеспечивающих универсальное поведение всех объектов.

1. toString()

Возвращает строковое представление объекта.

```
public String toString()
```

По умолчанию возвращает имя класса и хеш-код объекта, но часто **переопределяется** для удобного вывода информации

2. equals(Object obj)

Используется для сравнения объектов на логическое равенство.

```
public boolean equals(Object obj)
```

По умолчанию сравнивает ссылки (==), но может быть переопределён для сравнения содержимого объектов.

3. hashCode()

Возвращает целочисленный хеш-код объекта.

```
public int hashCode()
```

Используется в хеш-коллекциях (HashMap, HashSet).

При переопределении equals() **обязательно** переопределять hashCode().

4. getClass()

Возвращает объект типа Class, представляющий фактический класс объекта во время выполнения.

```
public final Class<?> getClass()
```

Метод является final и не может быть переопределён.

5. clone()

Создаёт копию объекта.

```
protected Object clone() throws CloneNotSupportedException
```

Для использования требуется:

- реализация интерфейса Cloneable;
- переопределение метода.

7. Методы синхронизации потоков Используются в многопоточном программировании:

```
wait(), notify(), notifyAll()
```

Эти методы обеспечивают взаимодействие потоков через механизм мониторов.

Значение класса Object

Класс Object:

- обеспечивает единый интерфейс для всех объектов;
- поддерживает полиморфизм;
- служит основой для сравнения, вывода и синхронизации объектов.

6. Ключевое слово **static**. Ключевое слово **final**.

В языке Java ключевые слова `static` и `final` используются для управления временем жизни объектов, принадлежностью членов классов и возможностью их изменения. Они являются важными средствами объектно-ориентированного программирования.

Ключевое слово `static`

Назначение **static**

Ключевое слово **`static`** обозначает, что поле, метод или вложенный класс **принадлежит классу в целом**, а не конкретному объекту.

Особенности **static**

- создаётся в единственном экземпляре при загрузке класса;
- доступен без создания объекта;
- общий для всех экземпляров класса.

static-поля

Используются для хранения общих данных.

```
class Counter {  
    static int count = 0;  
}
```

Все объекты класса `Counter` используют одну и ту же переменную `count`.

static-методы

Методы, не зависящие от состояния конкретного объекта.

```
class MathUtils {  
    static int square(int x) {  
        return x * x;  
    }  
}
```

Ограничения:

- `static`-методы не могут напрямую обращаться к нестатическим полям и методам;
- не используют ссылку `this`.

static-блоки

Используются для инициализации статических полей.

```
static {  
    // выполняется один раз при загрузке класса  
}
```

static и main

Метод `main` объявляется как `static`, поскольку вызывается JVM без создания объекта класса.

Ключевое слово `final`

Назначение final

Ключевое слово **final** используется для запрета изменения:

- значения переменной;
- реализации метода;
- структуры класса.

final-переменные

- значение присваивается один раз;
- после инициализации изменить нельзя.

```
final int MAX = 100;
```

Для ссылочных типов:

- нельзя изменить ссылку;
- состояние объекта может изменяться.

final-методы

- не могут быть переопределены в подклассах;
- используются для предотвращения изменения логики метода.

```
public final void print() {  
}
```

final-классы

- не могут иметь подклассов;
- используются для обеспечения неизменности и безопасности.

Пример: класс `String`.

Совместное использование `static` и `final`

Часто используется для объявления **констант**:

```
public static final double PI = 3.1415926535;
```

Характеристики:

- принадлежит классу;
- не изменяется;
- доступна без создания объекта.

7. Методы. Статические методы и атрибуты.

Метод — это именованный блок кода класса, предназначенный для выполнения определённых действий и работы с данными объекта или класса.

Методы используются для:

- определения поведения объектов;
- обработки данных;
- реализации логики программы.

Общий вид метода:

```
модификаторы тип_возврата имя_метода(параметры) {  
    тело_метода}
```

Виды методов в Java

В Java методы делятся на:

- **нестатические (экземплярные)** — работают с конкретным объектом;
- **статические** — принадлежат классу в целом.

Статические методы

Статический метод объявляется с ключевым словом `static` и принадлежит классу, а не объекту.

Пример:

```
class MathUtils {  
    static int max(int a, int b) {  
        return a > b ? a : b; }  
}
```

Вызов:

```
int m = MathUtils.max(3, 5);
```

Особенности статических методов

- вызываются без создания объекта;
- не имеют доступа к нестатическим полям и методам напрямую;
- не используют ссылку `this`;
- часто применяются для вспомогательных функций.

Статические атрибуты (поля)

Статический атрибут — это поле класса, общее для всех его объектов.

Пример:

```
class Counter {  
    static int count = 0;  
  
    Counter() {  
        count++; }  
}
```

Все экземпляры класса `Counter` используют одно и то же поле `count`.

Использование статических методов и атрибутов

Статические методы и атрибуты применяются, когда:

- логика не зависит от состояния конкретного объекта;
- требуется общее состояние для всех объектов;
- необходимо сократить использование памяти.

Примеры:

- математические функции (`Math`);
- счётчики объектов;
- константы (`public static final`).

8. Передача объектов в методы. Перегрузка и переопределение методов.

В языке Java при вызове метода **все параметры передаются по значению**.

Для объектов это означает, что в метод передаётся **копия ссылки на объект**, а не сам объект. Следствия:

- метод может изменять состояние объекта;
- метод не может изменить саму ссылку вызывающего кода.

Пример:

```
class Person {
    String name; }
void changeName(Person p) {
    p.name = "Alex"; } // состояние изменится
void changeRef(Person p) {
    p = new Person(); } // ссылка изменится только локально
```

Перегрузка методов (method overloading) — это объявление нескольких методов с одинаковым именем, но разными списками параметров.

Различия могут быть:

- по количеству параметров;
- по типам параметров;
- по их порядку.

Пример:

```
class Calculator {
    int sum(int a, int b) {
        return a + b; }
    double sum(double a, double b) {
        return a + b; }}
```

Особенности перегрузки:

- выполняется на этапе компиляции;
- не зависит от возвращаемого типа;
- может использоваться для повышения читаемости кода.

Переопределение методов (method overriding) — это замена реализации метода суперкласса в подклассе.

Условия переопределения:

- одинаковая сигнатура метода;
- уровень доступа не может быть уже, чем у метода суперкласса;
- возвращаемый тип должен совпадать или быть ковариантным;
- метод суперкласса не должен быть `final`.

Пример:

```
class Animal {
    void makeSound() {
        System.out.println("Some sound"); }}
class Dog extends Animal {
    @Override
    void makeSound() {
        System.out.println("Bark"); }}
```

Особенности переопределения:

- определяется во время выполнения программы;
- является основой полиморфизма;
- используется аннотация `@Override` для контроля корректности.

9. Использование **super** и **this**.

Ключевые слова **this** и **super** используются для обращения к текущему объекту и к элементам его суперкласса. Они играют важную роль в наследовании, инициализации объектов и устранении неоднозначностей.

Ключевое слово **this**

Назначение **this**

this — это ссылка на текущий объект класса.

Используется для:

- различения полей класса и параметров метода с одинаковыми именами;
- обращения к нестатическим полям и методам текущего объекта;
- вызова другого конструктора того же класса.

Применение **this**

1. Доступ к полям объекта

```
class Person {  
    String name;  
    Person(String name) {  
        this.name = name; }  
}
```

2. Вызов методов текущего объекта

```
this.print();
```

3. Вызов конструктора текущего класса

```
this(0, 0);
```

❗ Вызов **this()** должен быть **первой строкой конструктора**.

Ключевое слово **super**

Назначение **super**

super — это ссылка на объект суперкласса (родительского класса).

Используется для:

- доступа к полям и методам суперкласса;
- вызова конструктора суперкласса.

Применение **super**

1. Доступ к членам суперкласса

```
super.method();  
super.field;
```

Используется при сокрытии (hiding) или переопределении методов.

2. Вызов конструктора суперкласса

```
super(10);
```

Особенности:

- должен быть первой строкой конструктора;
- если не указан явно, компилятор вставляет **super()** автоматически;
- если у суперкласса нет конструктора без параметров, вызов **super(...)** обязателен.

Сравнение **this** и **super**

Критерий	this	super
На что указывает	Текущий объект	Объект суперкласса
Используется в	Нестатическом контексте	Нестатическом контексте
Вызов конструктора	this()	super()
Количество вызовов	Один	Один

Ограничения

- **this** и **super** нельзя использовать в **static**-контексте;
- **this()** и **super()** нельзя использовать вместе в одном конструкторе;
- оба должны быть первой строкой конструктора.

10. Абстрактные классы и методы.

Абстрактный класс — это класс, объявленный с использованием ключевого слова `abstract`, который:

- может содержать абстрактные и обычные методы;
- **не может иметь экземпляров** (объекты абстрактного класса создать нельзя);
- используется как базовый класс для наследования.

Абстрактные классы применяются для описания **общего поведения группы родственных классов**, не определяя его полностью.

Пример:

```
abstract class Animal {  
    abstract void makeSound();  
}
```

Абстрактный метод — это метод без реализации, объявленный с ключевым словом `abstract`.

Особенности:

- не имеет тела метода;
- должен быть реализован в конкретных (неабстрактных) подклассах;
- может объявляться только в абстрактном классе.

Пример: `abstract void makeSound();`

Реализация абстрактных методов

Подкласс, наследующий абстрактный класс, обязан:

- либо реализовать **все** абстрактные методы;
- либо сам быть объявленным как `abstract`.

Пример:

```
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

Свойства абстрактных классов

Абстрактный класс может:

- содержать поля (в том числе `static` и `final`);
- иметь конструкторы;
- содержать реализованные методы;
- реализовывать интерфейсы.

Абстрактный класс **не может** быть объявлен как `final` и иметь экземпляры.

Конструкторы в абстрактных классах

Абстрактные классы могут иметь конструкторы, которые:

- вызываются при создании объектов подклассов;
- используются для инициализации полей базового класса.

Пример:

```
abstract class Shape {  
    int color;  
    Shape(int color) {  
        this.color = color;  
    }  
}
```

Назначение абстрактных классов

Абстрактные классы используются для:

- реализации принципа абстракции;
- описания общего интерфейса и частичной реализации;
- уменьшения дублирования кода;
- построения иерархий классов.

11. Внутренние (inner) классы.

Внутренний класс (inner class) — это класс, объявленный **внутри** другого класса или интерфейса.

Внутренние классы используются для логического объединения классов, которые тесно связаны между собой, и для повышения инкапсуляции

Виды внутренних классов

В Java различают **четыре типа** внутренних классов:

1. Внутренние классы экземпляра (non-static inner classes)
2. Статические вложенные классы (static nested classes)
3. Локальные классы
4. Анонимные классы

1. Внутренние классы экземпляра

Объявляются внутри класса **без** ключевого слова `static`.

Особенности:

- имеют доступ ко всем полям и методам внешнего класса, включая `private`;
- связаны с конкретным экземпляром внешнего класса;
- не могут содержать статические члены (кроме `static final` констант).

Пример:

```
class Outer {
    private int x = 10;

    class Inner {
        void print() {
            System.out.println(x);
        }
    }
}
```

Создание объекта:

```
Outer.Inner obj = new Outer().new Inner();
```

2. Статические вложенные классы

Объявляются с ключевым словом `static`.

Особенности:

- не связаны с экземпляром внешнего класса;
- имеют доступ только к статическим членам внешнего класса;
- используются как логически вложенные, но независимые классы.

Пример:

```
class Outer {
    static int x = 5;

    static class Nested {
        void print() {
            System.out.println(x);
        }
    }
}
```

3. Локальные классы

Объявляются внутри метода, конструктора или блока.

Особенности:

- доступны только в пределах блока;
- могут обращаться к `final` или эффективно `final` переменным метода;
- используются редко, для локальной логики.

Пример:

```
void method() {
    class Local {
        void show() {
            System.out.println("Local class");
        }
    }
}
```

4. Анонимные классы

Класс без имени, создаётся и используется **одновременно**.

Особенности:

- наследует класс или реализует интерфейс;
- используется для одноразовой реализации;
- не может иметь конструкторов.

Пример:

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Running");
    }
};
```

Преимущества использования внутренних классов

- логическая группировка классов;
- улучшение инкапсуляции;
- более читаемый и компактный код;
- удобство работы с обработчиками событий и интерфейсами.

Ограничения внутренних классов

- усложнение структуры программы;
- ограниченная область видимости;
- снижение читаемости при чрезмерном использовании.

12. Вложенные (nested) классы.

Вложенный класс (nested class) — это класс, объявленный **внутри** другого класса.

В Java термин *nested classes* охватывает **все классы, объявленные внутри класса**, однако на практике под вложенными классами чаще всего понимают **статические вложенные классы**.

Классификация вложенных классов

В Java различают:

- **статические вложенные классы (static nested classes);**
- **внутренние классы (inner classes)** — нестатические.

В данном вопросе основной акцент делается именно на **static nested classes**.

Статические вложенные классы

Определение

Статический вложенный класс — это класс, объявленный внутри другого класса с использованием ключевого слова `static`.

Пример:

```
class Outer {  
    static class Nested {  
        void show() {  
            System.out.println("Nested class");  
        }  
    }  
}
```

Особенности статических вложенных классов

- не связаны с экземпляром внешнего класса;
- имеют доступ **только к статическим членам** внешнего класса;
- могут содержать статические поля и методы;
- создаются без создания объекта внешнего класса.

Создание объекта: `Outer.Nested obj = new Outer.Nested();`

Назначение статических вложенных классов

Статические вложенные классы используются, когда:

- класс логически связан с внешним классом;
- вложенному классу **не нужен доступ к нестатическим полям** внешнего класса;
- требуется группировка вспомогательных классов.

Отличие вложенных и внутренних классов

Критерий	Вложенный (static)	Внутренний (inner)
Ключевое слово <code>static</code>	Да	Нет
Связь с объектом внешнего класса	Нет	Да
Доступ к нестатическим членам	Нет	Да
Создание объекта	Без внешнего объекта	Через внешний объект
Статические члены	Разрешены	Запрещены (кроме констант)

Преимущества вложенных классов

- логическая группировка классов;
- улучшение читаемости и структуры кода;
- уменьшение области видимости вспомогательных классов;
- отсутствие лишней зависимости от объекта внешнего класса.

Ограничения

- вложенный класс не может напрямую обращаться к нестатическим членам внешнего класса;
- чрезмерное использование может усложнять архитектуру программы.

13. Анонимные (anonymous) классы.

Анонимный класс — это **класс без имени**, который объявляется и создаётся **одновременно**. Используется для **одноразовой реализации**:

- интерфейсов,
- абстрактных классов,
- или конкретных классов с переопределением методов.

Основные характеристики

- Не имеет имени и конструкторов.
- Объявляется **внутри выражения**.
- Используется **одноразово**, не содержит статических членов (кроме `static final`).
- Может обращаться к **полям внешнего класса** и **локальным переменным метода**, если они `final` или эффективно `final`.

Применение

- **Многопоточность** (`Runnable`, `Thread`).
- **Обработчики событий**.
- Работа с **интерфейсами с одним методом**.
- В современных версиях Java часто заменяются **лямбда-выражениями**, но остаются полезными для сложных случаев.

Преимущества

- Компактность кода.
- Уменьшение количества вспомогательных классов.
- Удобно для простой одноразовой логики.

Недостатки

- Сложно читать при большой логике.
- Невозможность повторного использования.

Итог

Анонимные классы позволяют **создавать объекты с переопределением методов без отдельного класса**, сокращая код и упрощая реализацию одноразовых задач.

14. Интерфейсы.

Интерфейс — это ссылочный тип в языке Java, который описывает **набор методов**, определяющих поведение класса, **без указания их реализации**.

Интерфейсы используются для задания контракта, которому должны следовать классы.

Интерфейс объявляется с помощью ключевого слова `interface`.

Пример:

```
interface Flyable {  
    void fly();}
```

Основные свойства интерфейсов:

- не может иметь экземпляров;
- может содержать только объявления методов и константы;
- реализуется классами с помощью ключевого слова `implements`;
- поддерживает множественную реализацию.

Методы интерфейса

По умолчанию методы интерфейса: являются `public`; являются `abstract` (до Java 8).

Начиная с Java 8, интерфейс может содержать:

1. Абстрактные методы

```
void move();
```

2. Методы по умолчанию (`default`)

```
default void print() {  
    System.out.println("Default method"); }
```

3. Статические методы

```
static void info() {  
    System.out.println("Static method"); }
```

Все поля интерфейса: являются `public`; являются `static`; являются `final`.

Пример:

```
interface Constants {  
    int MAX = 100;}
```

Реализация интерфейса

Класс, реализующий интерфейс:

- обязан реализовать **все абстрактные методы** интерфейса;
- либо сам должен быть объявлен как `abstract`.

Пример:

```
class Bird implements Flyable {  
    @Override  
    public void fly() {  
        System.out.println("Bird flies"); }}
```

Множественная реализация интерфейсов

В Java класс **не может наследоваться от нескольких классов**, но может реализовывать **несколько интерфейсов**.

Пример: `class Duck implements Flyable, Swimmable {}`

Это позволяет обойти ограничение множественного наследования классов.

Назначение интерфейсов

Интерфейсы используются для:

- реализации принципа абстракции;
- поддержки полиморфизма;
- задания общего контракта для несвязанных классов;
- уменьшения связанности компонентов системы.

15. Пакеты. Логические блоки.

Пакет (package) в Java — это механизм логической группировки классов, интерфейсов и под-пакетов.

Пакеты используются для:

- структурирования больших программ;
- предотвращения конфликтов имён классов;
- управления доступом к классам и их членам.

Пакет объявляется с помощью ключевого слова `package` и указывается **первой строкой файла**.

Пример:

```
package com.example.util;
```

Назначение пакетов как логических блоков

Пакеты выступают в роли **логических блоков программы**, объединяя:

- классы одной предметной области;
- функционально связанные компоненты;
- уровни архитектуры (model, service, controller).

Пример структуры:

```
com.example.app
├── model
├── service
└── controller
```

Стандартные пакеты Java

Java предоставляет множество встроенных пакетов, например:

- `java.lang` — базовые классы (подключается автоматически);
- `java.util` — коллекции и утилиты;
- `java.io` — ввод и вывод данных;
- `java.net` — работа с сетью.

Использование пакетов

Импорт классов

Для доступа к классам из другого пакета используется `import`.

```
import java.util.List;
```

Или:

```
import java.util.*;
```

Полные имена классов

При отсутствии `import` используется полное имя:

```
java.util.Scanner sc = new java.util.Scanner(System.in);
```

Пакеты и модификаторы доступа

Пакеты тесно связаны с механизмом ограничения доступа:

- `public` — доступен из любого пакета;
- `protected` — доступен в пределах пакета и в подклассах;
- **`package-private`** — доступен только внутри пакета;
- `private` — доступен только внутри класса.

Пользовательские пакеты

Пользователь может создавать собственные пакеты для организации кода и соблюдения архитектуры проекта.

Рекомендуется:

- использовать доменное имя в обратном порядке;

- избегать коротких и бессмысленных названий.

Пример:

```
package org.university.project.module;
```

Логические блоки в Java

Под **логическими блоками** обычно понимают:

- пакеты;
- классы;
- интерфейсы;
- модули (в более современных версиях Java).

Пакеты являются основным средством логического разделения кода до уровня модулей.

Преимущества использования пакетов

- упорядочивание структуры проекта;
- повышение читаемости и поддержки кода;
- уменьшение конфликтов имён;
- улучшение контроля доступа.

16. Иерархия и способы обработки исключительных ситуаций.

Исключительная ситуация (exception) — это ошибка или нестандартное состояние, возникающее во время выполнения программы и нарушающее её нормальный ход.

В Java обработка исключений является встроенным механизмом, позволяющим:

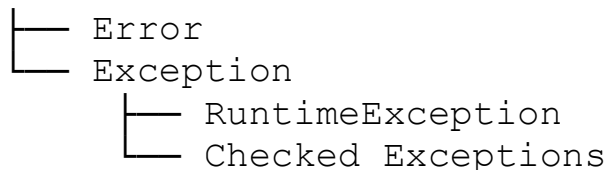
- обнаруживать ошибки во время выполнения;
- корректно реагировать на них;
- предотвращать аварийное завершение программы.

Иерархия исключений в Java

Все исключения в Java образуют иерархию, корнем которой является класс **Throwable**.

Основные ветви иерархии

Throwable



1. Error

Класс **Error** представляет **критические ошибки среды выполнения**, которые, как правило, невозможно или нецелесообразно обрабатывать программно.

Примеры:

- `OutOfMemoryError`
- `StackOverflowError`

❗ Обычно обработка **Error** не выполняется.

2. Exception

Класс **Exception** предназначен для описания ошибок, которые **могут и должны быть обработаны программой**.

Проверяемые исключения (Checked Exceptions)

- Проверяются на этапе компиляции;
- Обязаны быть обработаны или объявлены в сигнатуре метода.

Примеры:

- `IOException`
- `SQLException`
- `FileNotFoundException`

Непроверяемые исключения (RuntimeException)

- Наследуются от `RuntimeException`;
- Не проверяются компилятором;
- Обычно являются следствием логических ошибок в программе.

Примеры:

- `NullPointerException`
- `ArithmeticException`
- `ArrayIndexOutOfBoundsException`

Способы обработки исключений

1. Использование **try-catch**

Основной механизм обработки исключений.

```
try {
    int a = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Ошибка деления на ноль");
}
```

```
}
```

Можно использовать несколько блоков `catch`.

2. Блок **finally**

Блок `finally` выполняется **всегда**, независимо от того, было ли выброшено исключение.

Используется для:

- освобождения ресурсов;
- закрытия файлов и соединений.

```
finally {  
    // освобождение ресурсов  
}
```

3. Ключевое слово **throw**

Используется для **явного выбрасывания исключения**.

```
throw new IllegalArgumentException("Неверный аргумент");
```

4. Ключевое слово **throws**

Используется для **передачи ответственности за обработку исключения** вызывающему методу.

```
void readFile() throws IOException {  
}
```

5. Пользовательские исключения

Программист может создавать собственные классы исключений, наследуясь от `Exception` или `RuntimeException`.

```
class MyException extends Exception {  
    MyException(String message) {  
        super(message);  
    }  
}
```

6. Try-with-resources

Используется для автоматического закрытия ресурсов.

```
try (FileReader fr = new FileReader("file.txt")) {  
    // работа с файлом  
}
```

Назначение обработки исключений

Механизм исключений позволяет:

- отделить логику обработки ошибок от основной логики программы;
- повысить надёжность и устойчивость приложений;
- обеспечить корректное завершение работы программы.

17. Перехват и обработка исключения в блоке try-catch. Вложенные операторы
Перехват и обработка исключений — это механизм в Java, позволяющий программе корректно реагировать на ошибки, возникающие во время выполнения, и предотвращать аварийное завершение.

Блоки для обработки ошибок включают:

- **try** — блок кода, в котором может возникнуть исключение;
- **catch** — блок(и) для перехвата и обработки конкретных типов исключений;
- **finally** — необязательный блок, который выполняется всегда (для освобождения ресурсов).

Синтаксис блока try-catch

```
try {  
    // код, который может вызвать исключение  
    int a = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Ошибка: деление на ноль");  
} finally {  
    System.out.println("Блок finally выполнен");  
}
```

- Если в блоке try возникает исключение, управление передаётся **соответствующему блоку catch**.
- Если блок catch отсутствует или не совпадает с типом исключения — программа завершится с ошибкой.
- Блок finally выполняется **в любом случае**, независимо от наличия исключений.

Особенности блока catch

- Можно использовать **несколько блоков catch** для разных типов исключений:

```
try {  
    int[] arr = new int[5];  
    arr[10] = 3;  
} catch (ArithmeticException e) {  
    System.out.println("Деление на ноль");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Выход за пределы массива");  
}
```

- В Java 7+ можно объединять несколько типов исключений в одном catch:

```
catch (IOException | SQLException e) {  
    e.printStackTrace();  
}
```

Вложенные операторы try

Понятие

Вложенные try-операторы — это блоки try внутри других блоков try.

Используются для более точной локализации ошибок и индивидуальной обработки исключений в разных частях кода.

Пример вложенного try

```
try {  
    int[] arr = new int[5];  
    try {  
        arr[10] = 3;
```

```
    } catch (ArrayIndexOutOfBoundsException e) {  
        System.out.println("Внутренний catch: выход за пределы  
массива");  
    }  
  
    int a = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Внешний catch: деление на ноль");  
}
```

Пояснение:

- Исключение `ArrayIndexOutOfBoundsException` перехватывается **внутренним блоком `catch`**.
- Исключение `ArithmeticException` перехватывается **внешним блоком `catch`**.

Преимущества вложенных **try**

- Позволяют обрабатывать разные ошибки отдельно;
- Повышают читаемость и точность управления исключениями;
- Удобны при работе с ресурсами, требующими пошаговой обработки

Основные правила

- Блок `catch` должен **следовать сразу после блока `try`**;
- Блок `finally` может присутствовать с любым `try`;
- Вложенные блоки могут использовать свои собственные `catch` и `finally`;
- Внутренний блок `try` обрабатывает исключение локально, если нет подходящего `catch` — исключение передаётся внешнему блоку.

18. Оператор throw. Ключевое слово finally.

throw используется для **явного выбрасывания исключения** во время выполнения программы.

Синтаксис: `throw new ExceptionType ("Сообщение об ошибке") ;`

Пример:

```
void checkAge(int age) {  
    if (age < 18) {  
        throw new IllegalArgumentException("Возраст должен быть  
>= 18");  
    }  
}
```

Особенности: Используется для сигнализации о возникшей ошибке; Можно выбрасывать как стандартные исключения (`RuntimeException`, `IOException`), так и пользовательские; После **throw** выполнение текущего метода **прерывается**.

finally — это блок кода, который **выполняется всегда**, независимо от того, возникло исключение или нет.

Синтаксис:

```
try {  
    // код, который может вызвать исключение  
} catch (Exception e) {  
    // обработка исключения  
} finally {  
    // выполняется всегда }
```

Пример:

```
try {  
    int a = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Деление на ноль");  
} finally {  
    System.out.println("Блок finally выполнен");  
}
```

Особенности:

- Используется для освобождения ресурсов (заккрытие файлов, соединений с базой, сокетов и т.д.);
- Может идти после блока `try` с одним или несколькими `catch`;
- Выполняется даже если в блоке `try` был `return`.

Взаимодействие throw и finally. Если внутри блока `try` произошло **throw**, выполнение сначала перейдёт в **finally**, а затем исключение будет передано обработчику. Если в **finally** есть `return`, исключение будет подавлено (не рекомендуется использовать).

Пример:

```
try {  
    throw new RuntimeException("Ошибка");  
} finally {  
    System.out.println("finally выполнен");  
}
```

Вывод: `finally` выполнен. После чего исключение передается дальше.

Назначение

- `throw` — сообщает о возникшей ошибке;
- `finally` — гарантирует выполнение важного кода, например, освобождение ресурсов, независимо от исключений.

19. Оператор throws. Собственные исключения.

Оператор throws и собственные исключения в Java

Оператор throws

throws используется в сигнатуре метода для **объявления исключений**, которые может выбросить метод, но которые **не обрабатываются внутри метода**.

Синтаксис:

```
тип_возврата имяМетода(параметры) throws ExceptionType1,
ExceptionType2 {
    // код метода
}
```

Пример:

```
void readFile(String filename) throws IOException {
    FileReader fr = new FileReader(filename);
    // работа с файлом
}
```

Особенности:

- Сообщает вызывающему коду, что метод может выбросить исключение;
- Используется **только для проверяемых исключений (Checked Exceptions)**;
- Если метод выбрасывает RuntimeException, объявление throws не обязательно.

Передача исключений вызывающему коду

Использование throws позволяет:

- Делегировать обработку исключений вызывающему методу;
- Разделять ответственность за обработку ошибок между уровнями программы.

Пример:

```
void process() throws IOException {
    readFile("file.txt");
}
```

Собственные (пользовательские) исключения

Пользовательские исключения — это классы, создаваемые программистом для описания специфических ошибок программы.

Правила создания:

1. Наследовать класс от Exception (для проверяемых) или RuntimeException (для непроверяемых).
2. Определить конструкторы для передачи сообщений об ошибке.

Пример проверяемого пользовательского исключения:

```
class MyException extends Exception {
    MyException(String message) {
        super(message);
    }
}
```

```
void checkValue(int value) throws MyException {
    if (value < 0) {
        throw new MyException("Значение не может быть отрицательным");
    }
}
```

Пример непроверяемого исключения:

```

class MyRuntimeException extends RuntimeException {
    MyRuntimeException(String message) {
        super(message);
    }
}

```

Отличие проверяемых и непроверяемых пользовательских исключений

Критерий	Checked Exception	Unchecked Exception
Наследуется от	Exception	RuntimeException
Проверка компилятором	Да	Нет
Использование throws	Обязательно	Не обязательно
Применение	Ошибки, которые можно предвидеть	Логические ошибки

Назначение собственных исключений

- Отражают специфику конкретного приложения;
- Повышают читаемость и понятность кода;
- Позволяют точно обрабатывать ошибки.

20. Общее представление о потоках ввода/вывода.

В Java все операции **ввода и вывода данных** основаны на концепции **потоков (streams)**. Поток — это абстракция последовательности данных, которая позволяет **последовательно читать или записывать информацию** независимо от того, откуда она поступает или куда направляется. Можно представить поток как «трубу», по которой данные «текут»: входные потоки приносят данные в программу, выходные — отправляют их наружу.

Виды потоков

Потоки классифицируются по двум признакам:

1. По направлению:

- **Входные (InputStream / Reader)** — читают данные в программу;
- **Выходные (OutputStream / Writer)** — выводят данные из программы.

2. По типу данных:

- **Байтовые потоки (byte streams)** — работают с сырыми байтами, подходят для любых файлов, включая текст, изображения, звук и видео. Классы: `InputStream`, `OutputStream`.
- **Символьные потоки (character streams)** — работают с символами и текстом, поддерживая `Unicode`. Классы: `Reader`, `Writer`.

Дополнительно потоки могут быть **буферизованными**, что повышает производительность, уменьшая количество операций чтения и записи.

Основные классы потоков

- **Файловые потоки:** `FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter` — позволяют работать с файлами на диске.
- **Буферизованные потоки:** `BufferedReader`, `BufferedWriter` — ускоряют работу с данными, обеспечивая чтение и запись блоками.
- **Специальные потоки:** `PrintWriter` для удобного вывода текста, `DataInputStream` и `DataOutputStream` для работы с примитивными типами в бинарном виде.

Особенности работы с потоками

1. Потоки **нужно закрывать** после использования, чтобы освободить ресурсы, такие как файловые дескрипторы.
2. Для автоматического закрытия удобно использовать конструкцию **try-with-resources**, которая гарантирует закрытие потоков даже при возникновении исключений:

```
try (FileReader fr = new FileReader("file.txt")) {  
    // работа с файлом  
}
```

3. Потоки обеспечивают **последовательный доступ к данным**, что позволяет работать с большими файлами и потоками данных, не загружая их полностью в память.

Применение потоков

Потоки позволяют решать множество задач:

- читать и записывать текстовые и бинарные файлы,
- передавать данные по сети,
- создавать цепочки обработки данных (например, фильтры, шифраторы, декодеры),
- управлять ресурсами программы безопасно и эффективно.

Таким образом, потоки создают основу для работы с любыми данными, делая код **гибким, переносимым и безопасным**.

21. Байтовые потоки ввода.

Байтовые потоки ввода (InputStream) — это потоки, которые позволяют **считывать данные из источника в виде байтов**. Они используются, когда нужно работать с **любым типом данных**: текст, изображения, звук, видео или бинарные файлы.

Байтовый поток абстрагирует источник данных, позволяя программе не заботиться о том, откуда поступают данные — с диска, с сети или из памяти.

Основной класс

В Java базовым классом для всех байтовых потоков ввода является **InputStream**.

- Это **абстрактный класс**, который определяет методы для чтения данных:
 - `int read()` — считывает один байт и возвращает его в виде `int`;
 - `int read(byte[] b)` — считывает массив байт;
 - `void close()` — закрывает поток и освобождает ресурсы.

Основные производные классы

- **FileInputStream** — чтение данных из файла;
- **ByteArrayInputStream** — чтение данных из массива байтов в памяти;
- **BufferedInputStream** — буферизация потока, ускоряет чтение больших объёмов данных;
- **DataInputStream** — позволяет считывать примитивные типы (`int`, `float`, `double`) в бинарном виде.

Пример использования FileInputStream

```
try (FileInputStream fis = new FileInputStream("file.bin")) {  
    int b;  
    while ((b = fis.read()) != -1) {    // читаем байт за байтом  
        System.out.print((char)b);    // преобразуем байт в  
СИМВОЛ  
    }  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Пояснение:

- Метод `read()` возвращает **-1**, когда данные заканчиваются;
- Использование **try-with-resources** автоматически закрывает поток.

Особенности байтовых потоков

1. **Работают с любыми данными**, не только с текстом.
2. Чтение может выполняться **побайтово или блоками** для повышения скорости.
3. Для эффективной работы с файлами больших размеров часто применяют **буферизацию** через `BufferedInputStream`.
4. Обязательно **закрывать поток** после использования, чтобы не оставлять открытые ресурсы.

Применение

Байтовые потоки ввода используются, когда нужно:

- читать **бинарные файлы** (изображения, видео, архивы),
- обрабатывать данные с сетевых соединений,
- реализовать собственные форматы хранения данных,
- работать с файлами любого типа независимо от кодировки.

22. Байтовые потоки вывода.

Байтовые потоки вывода (OutputStream) — это потоки, которые позволяют **записывать данные в виде байтов** в различные источники: файлы, сетевые соединения или массивы в памяти.

Байтовые потоки подходят для работы с **любым типом данных**, включая текст, изображения, звук, видео и бинарные файлы, и позволяют абстрагироваться от конкретного носителя данных.

Основной класс

Базовым классом всех байтовых потоков вывода является **OutputStream**.

- Это **абстрактный класс**, который определяет методы для записи данных:
 - `void write(int b)` — записывает один байт;
 - `void write(byte[] b)` — записывает массив байт;
 - `void close()` — закрывает поток и освобождает ресурсы;
 - `void flush()` — принудительно выводит данные из буфера (важно при буферизации).

Основные производные классы

- **FileOutputStream** — запись данных в файл;
- **ByteArrayOutputStream** — запись данных в массив байт в памяти;
- **BufferedOutputStream** — буферизация потока, повышает производительность при записи больших объёмов данных;
- **DataOutputStream** — позволяет записывать примитивные типы (`int`, `double`, `float`) в бинарном виде.

Пример использования FileOutputStream

```
try (FileOutputStream fos = new FileOutputStream("file.bin")) {  
    String text = "Hello, Java!";  
    fos.write(text.getBytes()); // преобразуем строку в массив  
байт  
    fos.flush(); // гарантируем, что все данные  
записаны  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

Пояснение: Метод `write()` записывает данные в поток;

- Метод `flush()` обеспечивает запись всех данных из буфера;
- Конструкция `try-with-resources` автоматически закрывает поток.

Особенности байтовых потоков вывода

1. **Универсальность:** подходят для текстовых и бинарных данных.
2. **Буферизация:** через `BufferedOutputStream` запись больших объёмов данных становится быстрее.
3. **Контроль ресурсов:** всегда необходимо закрывать поток после использования, чтобы освободить системные ресурсы.
4. **Методы flush и close:** `flush()` выводит данные из буфера, а `close()` закрывает поток и очищает буфер.

Применение: Байтовые потоки вывода применяются для:

- записи **бинарных файлов** (изображения, видео, архивы);
- сохранения текстовых данных в нужной кодировке через преобразование в байты;
- передачи данных по сети; работы с собственными форматами данных;
- создания цепочек потоков для фильтрации и обработки данных.

23. Символьные потоки ввода.

Символьные потоки ввода (Reader) — это потоки, которые позволяют **считывать данные из источника в виде символов**.

Они предназначены для работы с **текстовой информацией**, поддерживают кодировку Unicode и абстрагируют источник данных — будь то файл, консоль или память.

Символьные потоки позволяют **удобно обрабатывать текст**, не заботясь о переводе байтов в символы вручную, что необходимо при работе с текстовыми файлами на разных языках.

Основной класс

Базовым абстрактным классом для всех символьных потоков ввода является **Reader**.

Основные методы:

- `int read()` — считывает один символ и возвращает его в виде `int`;
- `int read(char[] cbuf)` — считывает массив символов;
- `void close()` — закрывает поток и освобождает ресурсы;

Основные производные классы

- **FileReader** — чтение текста из файлов;
- **BufferedReader** — буферизация, ускоряет чтение больших объёмов текста и предоставляет метод `readLine()` для удобного построчного чтения;
- **CharArrayReader** — чтение текста из массива символов;
- **InputStreamReader** — позволяет преобразовать байтовый поток в символьный с нужной кодировкой, например UTF-8.

Пример использования **FileReader** и **BufferedReader**

```
try (BufferedReader br = new BufferedReader(new
FileReader("file.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {    // читаем построчно
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Пояснение:

- Метод `readLine()` удобно считывает строки текста;
- Конструкция `try-with-resources` автоматически закрывает поток;
- **BufferedReader** использует буфер, что повышает производительность при больших файлах.

Особенности символьных потоков ввода

1. Работают только с текстом (символами).
2. Поддерживают различные кодировки через **InputStreamReader**.
3. Буферизация улучшает производительность при больших объёмах текста.
4. Требуется **закрывать поток** после использования, чтобы освободить ресурсы.

Символьные потоки ввода применяются для:

- чтения текстовых файлов (например, `.txt`, `.csv`, `.xml`),
- обработки пользовательского ввода с консоли,
- работы с текстовыми данными в памяти,
- реализации парсеров, анализаторов и фильтров текста.

24. Символьные потоки вывода.

Символьные потоки вывода (Writer) — это потоки, которые позволяют **записывать текстовые данные в виде символов**.

Они предназначены для работы с **текстовой информацией**, поддерживают кодировку Unicode и абстрагируют источник данных — файл, консоль, память или сетевое соединение.

Символьные потоки удобны, когда нужно **обрабатывать текст**, не заботясь о преобразовании символов в байты.

Базовым абстрактным классом всех символьных потоков вывода является **Writer**.

Основные методы:

- `void write(int c)` — записывает один символ;
- `void write(char[] cbuf)` — записывает массив символов;
- `void write(String str)` — записывает строку текста;
- `void flush()` — выводит данные из буфера в источник;
- `void close()` — закрывает поток и освобождает ресурсы.

Основные производные классы

- **FileWriter** — запись текста в файл;
- **BufferedWriter** — буферизация, повышает скорость записи и предоставляет метод `newLine()` для перехода на новую строку;
- **CharArrayWriter** — запись текста в массив символов в памяти;
- **PrintWriter** — удобный вывод текста с форматированием, поддерживает методы `print()` и `println()`.

Пример использования FileWriter и BufferedWriter

```
try (BufferedWriter bw = new BufferedWriter(new
FileWriter("file.txt"))) {
    bw.write("Hello, Java!");
    bw.newLine(); // переход на новую строку
    bw.write("Символьные потоки вывода");
    bw.flush();    // гарантирует, что данные записаны
} catch (IOException e) {
    e.printStackTrace();}
```

Пояснение:

- Метод `write()` записывает символы или строки;
- Метод `flush()` выводит данные из буфера;
- **BufferedWriter** повышает производительность при больших объёмах текста;
- `try-with-resources` автоматически закрывает поток.

Особенности символьных потоков вывода

1. Работают только с текстом (символами).
2. Поддерживают различные кодировки через `OutputStreamWriter`.
3. Буферизация ускоряет запись больших текстов.
4. Всегда важно закрывать поток после работы, чтобы освободить ресурсы и записать все данные.

Символьные потоки вывода используются для:

- записи текстовых файлов (`.txt`, `.csv`, `.xml`),
- вывода на консоль, лог-файлы или сетевые соединения,
- генерации текстовых отчётов и форматов данных,
- реализации фильтров и преобразователей текста.

25. Классы `PrintStream` и `PrintWriter`.

`PrintStream` и `PrintWriter` — это классы для **вывода данных в удобной текстовой форме**. Они позволяют выводить данные **в поток символов или байтов** с возможностью форматирования и автоматической конвертации типов (`int`, `double`, `boolean`, `String`) в текст.

Эти классы часто используются для:

- вывода на консоль,
- записи в текстовые файлы,
- формирования логов и отчётов.

Класс `PrintStream`

- Работает с **байтовыми потоками вывода** (`OutputStream`).
- Автоматически **преобразует любые типы данных в строку**.
- Предоставляет методы:
 - `print()` — вывод без переноса строки;
 - `println()` — вывод с переходом на новую строку;
 - `printf()` — форматированный вывод.
- Не генерирует исключения `IOException` при записи, что упрощает работу.

Пример использования:

```
PrintStream ps = new PrintStream(System.out);
ps.println("Hello, Java!");
ps.printf("Число: %d, текст: %s\n", 42, "пример");
ps.close();
```

Класс `PrintWriter`

- Работает с **символьными потоками** (`Writer`) или с байтовыми потоками через адаптер.
- Предоставляет те же методы, что и `PrintStream`: `print()`, `println()`, `printf()`.
- В отличие от `PrintStream`, может быть **буферизован** через `BufferedWriter`, что ускоряет запись в файлы.
- Может быть настроен на **автофлашинг** — автоматический вывод данных при вызове `println()`, `printf()` или `format()`.

Пример использования:

```
try (PrintWriter pw = new PrintWriter(new BufferedWriter(new
    FileWriter("output.txt")), true)) {
    pw.println("Hello, Java!");
    pw.printf("Число: %d, текст: %s\n", 42, "пример");
}
```

Отличия `PrintStream` и `PrintWriter`

Критерий	<code>PrintStream</code>	<code>PrintWriter</code>
Тип потока	Байтовый	Символьный
Работа с текстом	Преобразует автоматически	Преобразует автоматически
Исключения	Не выбрасывает <code>IOException</code>	Может выбрасывать <code>IOException</code> при работе с потоками
Буферизация	Нет (нужен <code>BufferedOutputStream</code>)	Есть (<code>BufferedWriter</code>)

Критерий	PrintStream	PrintWriter
Автофлашинг	Да (при println, printf)	Да (при println, printf и опционально при создании)

Применение

1. **PrintStream** — для быстрого вывода на консоль, например `System.out`.
2. **PrintWriter** — для записи текста в файлы или сетевые соединения с возможностью буферизации и автофлашинга.
3. Используются для удобного форматирования и вывода данных разных типов без необходимости ручного преобразования в строки.

26. Сериализация и десериализация.

Сериализация — это процесс преобразования объекта в последовательность байтов для последующего хранения или передачи.

Десериализация — обратный процесс: восстановление объекта из последовательности байтов.

Эти механизмы позволяют сохранять состояние объектов, передавать их по сети или записывать в файлы.

Назначение

- Сохранение объектов в файл для последующего использования;
- Передача объектов через сеть (например, при работе с RMI или сокетами);
- Кэширование объектов в памяти или на диске;
- Копирование объектов путем сериализации и последующей десериализации.

Как сделать объект сериализуемым

1. Класс должен **impleментировать интерфейс Serializable**:

```
import java.io.Serializable;

class Person implements Serializable {
    private String name;
    private int age;

    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

2. Все поля класса должны быть сериализуемыми, иначе нужно пометить их ключевым словом `transient` (они не будут сериализоваться).

Основные классы для сериализации

- **ObjectOutputStream** — используется для записи сериализованных объектов в поток вывода (файл, сеть).
- **ObjectInputStream** — используется для чтения объектов из потока ввода.

Пример сериализации

```
import java.io.*;

Person person = new Person("Alice", 25);

try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("person.dat"))) {
    oos.writeObject(person); // сериализация объекта
} catch (IOException e) {
    e.printStackTrace();
}
```

Пример десериализации

```
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("person.dat"))) {
    Person person = (Person) ois.readObject(); // десериализация
    объекта
}
```

```
        System.out.println(person.getName() + ", " +  
person.getAge());  
    } catch (IOException | ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
}
```

Особенности сериализации

1. **Класс должен быть сериализуемым** (Serializable).
2. **Поля `transient` не сохраняются**, используются для временных или чувствительных данных.
3. **Сериализация сохраняет состояние объекта**, включая значения полей, но **не методы**.
4. **Нельзя сериализовать объекты с несериализуемыми полями**, если они не помечены `transient`.

Применение

- Сохранение состояния объектов в файл для последующего восстановления;
- Передача объектов по сети (RMI, сокет);
- Кэширование данных;
- Глубокое копирование объектов через сериализацию и десериализацию.

27. Класс File.

Класс **File** в Java представляет файл или директорию на диске и предоставляет методы для управления ими и получения информации о них. Важно понимать, что объект класса **File** **не хранит содержимое файла**, а лишь **описывает путь к нему** и позволяет выполнять операции с файловой системой.

Например, создание объекта **File** только указывает путь, но не создаёт сам файл:

```
File file = new File("data.txt");
```

С помощью класса **File** можно:

- **проверять существование файлов и папок**

```
file.exists();
```

- **различать файлы и директории**

```
file.isFile();
```

```
file.isDirectory();
```

- **создавать и удалять файлы или каталоги**

```
file.createNewFile();
```

```
file.delete();
```

- **получать информацию о файле** — имя, путь, размер, дату последнего изменения, родительскую директорию

```
file.getName();
```

```
file.getAbsolutePath();
```

```
file.length();
```

```
file.lastModified();
```

```
file.getParent();
```

- **получать список содержимого папки**

```
File dir = new File("folder");
```

```
File[] files = dir.listFiles();
```

Класс **File** поддерживает работу как с **относительными**, так и с

абсолютными путями. Он часто используется **совместно с потоками**

ввода/вывода для чтения и записи данных, выступая в роли объекта, который указывает, **с каким файлом необходимо работать**. Например, **File** передаётся в поток:

```
FileInputStream fis = new FileInputStream(file);
```

Таким образом, класс **File** является инструментом для **работы с файлами и директориями на уровне файловой системы**. Он предназначен не для обработки содержимого файлов, а для **организации, контроля и получения информации о файловой структуре**, и служит основой для дальнейшей работы с вводом и выводом данных в Java.

28. Класс Scanner.

Класс **Scanner** используется для **считывания данных из различных источников**, таких как консоль, файлы или строки. Он является удобным инструментом для ввода данных различного типа, включая числа, строки и отдельные токены, с автоматическим разбором данных по разделителям. Наиболее часто Scanner применяется для **взаимодействия с пользователем через консоль**, а также для чтения текстовых файлов.

Основные возможности класса Scanner

С помощью Scanner можно:

- **Считывать данные построчно или по отдельным токенам**

```
Scanner sc = new Scanner(System.in);  
String line = sc.nextLine();
```

- **Преобразовывать вводимые данные в примитивные типы**

Для этого используются специальные методы:

```
int x = sc.nextInt();  
double d = sc.nextDouble();  
boolean b = sc.nextBoolean();
```

- **Настраивать разделители ввода**

По умолчанию используется пробел, но его можно изменить:

```
sc.useDelimiter(",");
```

- **Работать с различными источниками данных**

```
Scanner console = new Scanner(System.in);  
Scanner fileScanner = new Scanner(new File("data.txt"));  
Scanner stringScanner = new Scanner("10 20 30");
```

Проверка наличия данных

Перед чтением данных рекомендуется проверять их наличие, чтобы избежать ошибок:

```
if (sc.hasNextInt()) {  
    int value = sc.nextInt();  
}
```

Закрытие Scanner

После завершения работы объект Scanner следует закрывать, чтобы освободить ресурсы:

```
sc.close();
```

(При работе с `System.in` закрывать Scanner нужно осторожно, так как поток может использоваться дальше.)

Особенности и ограничения

- Scanner **удобен, но не самый быстрый** способ ввода данных;
- Для больших объёмов данных чаще используют `BufferedReader`;
- Чувствителен к формату входных данных и может выбрасывать исключения при неправильном вводе.

29. Коллекции. Общая характеристика. Алгоритмы класса `Collections`.

Коллекции в Java — это объекты, предназначенные для **хранения, управления и обработки групп элементов**. Они позволяют работать с множествами данных без необходимости вручную управлять массивами и их размером.

Основные особенности коллекций:

- Могут хранить объекты любого типа;
- Позволяют автоматически изменять размер и управлять содержимым;
- Обеспечивают удобные методы для добавления, удаления, поиска и перебора элементов;
- Делятся на несколько типов:
 - **Списки (List)** — упорядоченные коллекции, допускающие дубликаты;
 - **Множества (Set)** — уникальные элементы без порядка;
 - **Очереди (Queue, Deque)** — элементы обрабатываются по принципу «первым пришёл — первым вышел» или двунаправленно;
 - **Словари (Map)** — хранят пары «ключ-значение», где ключи уникальны.

Коллекции находятся в пакете `java.util` и реализованы через интерфейсы и классы, обеспечивающие разные структуры данных и алгоритмы (например, `ArrayList`, `LinkedList`, `HashSet`, `TreeMap`).

Алгоритмы класса `Collections`

Класс `Collections` предоставляет **статические методы** для работы с коллекциями:

- **Сортировка:** `sort(list)` — сортирует список по естественному порядку или с использованием компаратора;
- **Поиск и проверка:** `binarySearch(list, key)` — бинарный поиск, `max()`, `min()`, `contains()`;
- **Изменение коллекций:** `reverse()`, `shuffle()` — изменение порядка элементов; `fill()` — заполнение коллекции одним значением;
- **Общая работа с элементами:** `copy()`, `swap()` — копирование и обмен элементов в списках.

Эти алгоритмы работают с **любым объектом, реализующим интерфейс `Collection` или `List`**, и позволяют быстро выполнять стандартные операции без написания собственного кода.

30. Списки. Иерархия классов.

Список (`List`) — это коллекция, в которой элементы упорядочены, индексируются и допускается хранение дубликатов.

Каждый элемент имеет индекс, что позволяет обращаться к элементам по позиции, а также вставлять и удалять элементы в любом месте списка.

Списки удобно использовать, когда важен порядок элементов и требуется доступ по индексу.

Интерфейс `List`

Интерфейс `List` находится в пакете `java.util` и расширяет интерфейс `Collection`.

Он определяет общий контракт для всех списков, что позволяет работать с разными реализациями (`ArrayList`, `LinkedList`) через единый интерфейс.

Пример объявления:

```
List<String> list = new ArrayList<>();
```

Основные реализации

`ArrayList`

Основан на массиве. Обеспечивает быстрый доступ по индексу, но операции вставки и удаления в середине списка выполняются медленно, так как требуют сдвига элементов.

`LinkedList`

Основан на двусвязном списке. Быстро выполняет вставку и удаление элементов, но медленно работает с доступом по индексу. Также может использоваться как очередь или стек, так как реализует `Queue` и `Deque`.

`Vector`

Синхронизированная версия `ArrayList`. Считается устаревшим и редко используется в современных приложениях.

`Stack`

Наследуется от `Vector` и реализует структуру стека. В новых проектах рекомендуется использовать `Deque`.

Основные возможности списков

Списки позволяют:

- добавлять, удалять и изменять элементы по индексу;
- получать элементы по индексу или через итератор;
- хранить дубликаты и сохранять порядок элементов;
- использовать алгоритмы класса `Collections` (сортировка, поиск, перемешивание).

Простой пример работы со списком

```
List<String> names = new ArrayList<>();
names.add("Anna");
names.add("Bob");
names.add("Anna");
```

```
names.remove(1);
System.out.println(names.get(0));
```

Дополнительные особенности

- Списки могут содержать значение `null`;
- Большинство реализаций `List` **не являются потокобезопасными**;
- Выбор реализации зависит от характера операций: доступ по индексу или частые вставки/удаления.

31. Множества. Иерархия классов.

Множество (`Set`) — это коллекция, которая хранит **уникальные элементы**, то есть дубликаты в ней запрещены.

Основное назначение множеств — хранение данных без повторений и быстрая проверка наличия элемента.

Порядок элементов, как правило, не гарантируется, за исключением некоторых реализаций.

Интерфейс `Set`

Интерфейс `Set` расширяет `Collection` и определяет контракт коллекции без дубликатов. Добавление элемента, уже существующего в множестве, не приводит к ошибке, но элемент не будет добавлен повторно.

Основные реализации

`HashSet`

Основан на хеш-таблице и обеспечивает очень быстрое добавление, удаление и поиск элементов. Порядок элементов не сохраняется.

`LinkedHashSet`

Расширяет `HashSet` и сохраняет порядок добавления элементов, при этом немного уступает по производительности.

`TreeSet`

Основан на красно-чёрном дереве и хранит элементы в отсортированном виде — либо по естественному порядку, либо с использованием компаратора.

Особенности работы

- Уникальность элементов определяется методами `equals()` и `hashCode()`;
- Большинство реализаций допускают хранение **одного значения `null`** (кроме `TreeSet`);
- Множества обычно не являются потокобезопасными.

Основные возможности

Множества позволяют:

- гарантировать уникальность элементов;
- быстро выполнять операции `add()`, `remove()` и `contains()`;
- использовать алгоритмы класса `Collections`;
- обходить элементы с помощью итераторов.

Короткий пример

```
Set<String> set = new HashSet<>();  
set.add("Java");  
set.add("Java");  
set.add("Python");
```

```
System.out.println(set.size()); // 2
```


32. Карты отображений. Иерархия классов.

Карты отображений в Java

Map — это структура данных, которая хранит элементы в виде пар «ключ–значение».

Каждый ключ в карте уникален и используется для быстрого доступа к соответствующему значению.

Карты широко применяются для реализации словарей, кэшей, таблиц соответствий и хранения данных по уникальному идентификатору.

Интерфейс Map не наследуется от Collection, так как имеет другую модель хранения данных. Основными реализациями являются HashMap, LinkedHashMap и TreeMap.

HashMap обеспечивает максимально быструю работу с данными, но не сохраняет порядок элементов.

LinkedHashMap сохраняет порядок добавления или доступа к элементам.

TreeMap хранит данные в отсортированном виде и позволяет выполнять навигационные и диапазонные операции.

Уникальность ключей в Map определяется методами equals() и hashCode(), что особенно важно при использовании хеш-таблиц.

Основные операции

Карты позволяют:

- добавлять и удалять элементы по ключу;
- получать значение по ключу;
- проверять наличие ключей и значений;
- перебирать элементы в виде пар «ключ–значение».

Короткий пример

```
Map<String, Integer> map = new HashMap<>();  
map.put("Java", 10);  
map.put("Python", 8);
```

```
int value = map.get("Java");
```

33. Универсальные классы и методы.

Универсальные классы и методы в Java

Универсальные классы и методы (Generics) позволяют писать код, работающий с различными типами данных, при этом сохраняя типовую безопасность. Тип данных передаётся в виде параметра, что позволяет избежать явных приведений типов и обнаруживать ошибки ещё на этапе компиляции.

Универсальные классы

Универсальный класс объявляется с параметром типа, который задаётся при создании объекта.

Это позволяет использовать один и тот же класс для разных типов данных без потери безопасности и читаемости кода.

```
class Box<T> {  
    private T value;  
    public void set(T value) { this.value = value; }  
    public T get() { return value; }  
}
```

Универсальные методы

Универсальные методы позволяют использовать обобщённые типы независимо от того, является ли сам класс универсальным.

Тип параметра задаётся непосредственно при вызове метода.

```
public <T> void printArray(T[] array) {  
    for (T element : array) {  
        System.out.println(element);  
    }  
}
```

Ограничения и особенности

- обобщённые типы работают только с объектами, примитивы используются через классы-обёртки;
- можно задавать ограничения типов с помощью `extends`;
- во время выполнения происходит стирание типов;
- Generics обеспечивают типовую безопасность на этапе компиляции.

Применение

Generics широко используются в коллекциях, универсальных классах-хранилищах и методах обработки данных, делая код гибким, безопасным и повторно используемым

34. Универсальные интерфейсы. Наследование и универсальные типы.

Универсальные интерфейсы — это интерфейсы с параметрами типа (**Generics**), которые позволяют **определять поведение для объектов разных типов**, сохраняя типовую безопасность.

Использование универсальных интерфейсов повышает гибкость кода и упрощает работу с обобщёнными коллекциями и методами.

Пример объявления универсального интерфейса

```
public interface Container<T> {  
    void add(T item);  
    T get(int index);  
}
```

- T — параметр типа, который определяется при реализации интерфейса;
- Интерфейс можно реализовать для любых типов данных:

```
class Box implements Container<String> { ... }
```

Наследование и универсальные типы

1. Наследование универсальных интерфейсов классами

Класс, реализующий универсальный интерфейс, может **указать конкретный тип**:

```
    class IntegerBox implements Container<Integer> { ... }
```

Или оставлять параметр типа открытым для дальнейшего использования:

```
    class GenericBox<T> implements Container<T> { ... }
```

2. Наследование универсальных интерфейсов интерфейсами

Универсальный интерфейс может расширять другой универсальный интерфейс, сохраняя или уточняя типы:

```
interface AdvancedContainer<T> extends Container<T> {  
    void remove(T item);  
}
```

3. Связь с Generics

Универсальные интерфейсы позволяют **создавать классы и методы, работающие с любыми типами данных**, поддерживая типовую безопасность на этапе компиляции; Это упрощает работу с коллекциями, обобщёнными методами и структурами данных.

Применение

Универсальные интерфейсы используются для создания:

- Определение **обобщённых контрактов** для классов и коллекций;
- Создание **универсальных библиотек и API**, совместимых с разными типами данных;
- Использование в связке с универсальными классами и методами для **повторного использования кода**;
- Поддержка **гибкого наследования и полиморфизма** для обобщённых типов.

35. Интерфейс Comparable. Интерфейс Comparator.

В языке Java для сравнения и сортировки объектов применяются интерфейсы **Comparable** и **Comparator**, которые определяют правила сравнения объектов и позволяют использовать их в алгоритмах сортировки и упорядоченных коллекциях.

Интерфейс **Comparable** задаёт **естественный порядок объектов**. Класс реализует его напрямую и переопределяет метод `compareTo`, который сравнивает текущий объект с другим и возвращает число: отрицательное, если текущий объект меньше, ноль — если объекты равны, и положительное — если текущий объект больше. Такой подход удобен, когда у объектов есть один логичный порядок, например числа по возрастанию или строки по алфавиту. Реализовав **Comparable**, объекты можно напрямую сортировать с помощью `Collections.sort(list)` или `Arrays.sort(array)`.

Интерфейс **Comparator** предназначен для задания **альтернативных вариантов сравнения** без изменения класса объектов. Он определяет метод `compare`, который сравнивает два объекта по нужному критерию. **Comparator** широко применяется, когда требуется сортировка по нескольким признакам, например по возрасту, имени или дате создания. Для удобства его можно реализовать через отдельный класс, анонимный класс или лямбда-выражение, а затем передавать в методы сортировки коллекций, например `Collections.sort(list, comparator)`.

Эти интерфейсы применяются не только при прямой сортировке, но и в упорядоченных коллекциях, таких как **TreeSet** и **TreeMap**, где порядок элементов определяется сравнением. При этом **Comparable** задаёт естественный порядок для объектов внутри коллекции, а **Comparator** позволяет использовать альтернативные критерии сортировки без изменения классов объектов.

Пример использования **Comparator** через лямбду:

```
Comparator<String> cmp = (a, b) -> a.length() -  
b.length();  
Collections.sort(listOfStrings, cmp);
```

Таким образом, **Comparable** обеспечивает естественный порядок объектов, а **Comparator** — гибкость для сортировки по разным критериям. Совместно эти интерфейсы позволяют эффективно управлять порядком элементов в коллекциях и сортировать данные в соответствии с требуемой логикой, сохраняя при этом типовую безопасность и удобство работы с объектами.

36. Интерфейс Iterator. Интерфейс ListIterator.

Интерфейсы `Iterator` и `ListIterator` в Java

`Iterator<E>` — интерфейс для перебора элементов коллекции. Основные методы:

- `hasNext()` — проверяет, есть ли следующий элемент
- `next()` — возвращает следующий элемент
- `remove()` — удаляет текущий элемент (опциональный)

`ListIterator<E>` — расширяет `Iterator` для работы со списками (`List`). Добавляет:

- Двухнаправленный обход (`hasPrevious()`, `previous()`)
- Получение индексов (`nextIndex()`, `previousIndex()`)
- Модификация списка (`set()`, `add()`)

Пример из жизни:

`Iterator` — как экскурсовод в музее, который ведет группу от экспоната к экспонату только вперед.

`ListIterator` — как гид, который может вести группу и вперед, и назад, а также комментировать экспонаты.

Код:

```
java
import java.util.*;

public class IteratorExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Первый");
        list.add("Второй");
        list.add("Третий");
        list.add("Четвертый");

        // 1. Использование Iterator
        System.out.println("Обход с Iterator (только вперед):");
        Iterator<String> iterator = list.iterator();

        while (iterator.hasNext()) {
            String element = iterator.next();
            System.out.println(element);

            // Удаление элемента во время итерации
            if (element.equals("Третий")) {
                iterator.remove(); // Безопасное удаление
            }
        }
        System.out.println("После удаления 'Третий': " + list);

        // 2. Использование ListIterator (двухнаправленный)
        System.out.println("\nОбход с ListIterator:");
        ListIterator<String> listIterator = list.listIterator();

        // Движение вперед
        while (listIterator.hasNext()) {
```

```

String element = listIterator.next();
System.out.println("Вперед: " + element + " (индекс: " +
    listIterator.nextIndex() + ")");

// Замена элемента
if (element.equals("Второй")) {
    listIterator.set("Измененный Второй");
}

// Добавление элемента
if (element.equals("Первый")) {
    listIterator.add("Новый между Первым и Вторым");
}
}

System.out.println("После изменений: " + list);

// Движение назад
System.out.println("\nДвижение назад:");
while (listIterator.hasPrevious()) {
    String element = listIterator.previous();
    System.out.println("Назад: " + element + " (предыдущий индекс: " +
        listIterator.previousIndex() + ")");
}

// 3. Начало с определенной позиции
ListIterator<String> middleIterator =

```

37. Лямбда-выражения.

Лямбда-выражения — это анонимные функции, которые позволяют передавать поведение (код) как аргумент метода или сохранять его в переменной. Они представляют собой реализацию функционального интерфейса (интерфейса с одним абстрактным методом) в краткой форме.

Ключевые характеристики:

1. Анонимность — не имеют имени
2. Функциональность — представляют собой функцию
3. Краткость — синтаксис более компактен, чем у анонимных классов
4. Передаваемость — могут быть переданы как аргумент метода или возвращены как результат

Синтаксис:

text

(параметры) -> выражение

(параметры) -> { операторы; }

Типы параметров можно опускать, если их можно вывести из контекста. Если параметр один — скобки можно опустить.

Пример из жизни:

Представьте, что вам нужно отфильтровать список студентов по возрасту. Вместо создания отдельного класса-фильтра вы можете написать краткое условие прямо в месте использования: `student -> student.getAge() > 18`.

Эволюция синтаксиса:

java

// Java 7: анонимный класс

```
Comparator<String> c1 = new Comparator<String>() {  
    @Override  
    public int compare(String a, String b) {  
        return a.compareTo(b);  
    }  
};
```

// Java 8: лямбда-выражение

```
Comparator<String> c2 = (a, b) -> a.compareTo(b);
```

Область видимости:

Лямбда-выражения имеют доступ к:

- Своим параметрам
- Полям класса, в котором объявлены
- Локальным переменным (если они `effectively final`)
- Статическим переменным

38. Функциональный интерфейс. Передача метода в качестве параметра.

Функциональный интерфейс — это интерфейс, который содержит **ровно один абстрактный метод**.

Он используется как **тип для лямбда-выражений, ссылок на методы и анонимных классов**.

- Аннотация `@FunctionalInterface` опциональна, но позволяет компилятору проверять, что интерфейс действительно функциональный.
- Примеры встроенных функциональных интерфейсов в Java:
 - `Predicate<T>` — проверка условия;
 - `Function<T, R>` — преобразование значения;
 - `Consumer<T>` — выполнение действия без возвращаемого значения;
 - `Supplier<T>` — поставка значения.

Передача метода в качестве параметра

- В Java можно **передавать поведение в виде метода** через:
 - **лямбда-выражения**,
 - **ссылки на методы** (`ClassName::methodName`),
 - **анонимные классы**.
- Это позволяет **делегировать выполнение задачи** вызывающему коду, делая методы более универсальными и гибкими.

Пример передачи метода через функциональный интерфейс:

```
public void executeOperation(Function<Integer, Integer>
operation, int value) {
    System.out.println(operation.apply(value));
}
```

```
// Вызов с лямбдой
executeOperation(x -> x * 2, 5);
```

```
// Вызов через ссылку на метод
executeOperation(Math::abs, -10);
```

- Метод `executeOperation` **не знает конкретной реализации операции** заранее; она передается извне.
- Это реализует принцип **передачи поведения как параметра** и используется в функциональном программировании.

Применение

- Работа с **Stream API**: фильтрация, преобразование, агрегация данных;
- Обработка событий в GUI и веб-приложениях;
- Создание универсальных и переиспользуемых методов и библиотек;
- Реализация обратного вызова (callback) и делегирования логики.

39. Функциональные интерфейсы: Predicate, Function, BinaryOperator.

Функциональный интерфейс — это интерфейс с одним абстрактным методом, который используется как цель для **лямбда-выражений** и **ссылок на методы**. Они позволяют передавать поведение как параметр, что упрощает работу с коллекциями, потоками данных и алгоритмами.

Java предоставляет большое количество стандартных функциональных интерфейсов в пакете `java.util.function`, которые активно применяются в **Stream API** и при работе с коллекциями. Основные из них:

1. Predicate<T>

Predicate представляет собой **функцию условия**, которая принимает объект типа `T` и возвращает `boolean`. Основной метод интерфейса — `boolean test(T t)`.

Применение: проверка условий, фильтрация коллекций.

Пример: фильтрация списка чисел, оставляя только чётные:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
List<Integer> even = numbers.stream()
    .filter(n -> n % 2 == 0)    //
    .collect(Collectors.toList());
System.out.println(even); // [2, 4]
```

2. Function<T, R>

Function представляет функцию преобразования объекта типа `T` в объект типа `R`. Основной метод — `R apply(T t)`.

Применение: преобразование или отображение элементов коллекции, например с помощью `map()`.

Пример: преобразование списка строк в длины этих строк:

```
List<String> words = Arrays.asList("Java", "Python", "C++");
List<Integer> lengths = words.stream()
    .map(s -> s.length())    //
    .collect(Collectors.toList());
System.out.println(lengths); // [4, 6, 3]
```

3. BinaryOperator<T>

BinaryOperator представляет операцию над двумя объектами одного типа и возвращает результат того же типа. Он наследует `BiFunction<T, T, T>`, а основной метод — `T apply(T t1, T t2)`.

Применение: объединение двух элементов коллекции, редукция, арифметические операции.

Пример: суммирование всех чисел списка с использованием `reduce()`:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int sum = numbers.stream()
    .reduce(0, (a, b) -> a + b); //
BinaryOperator<Integer>
System.out.println(sum); // 15
```

Дополнительная информация

1. Лямбда-выражения и ссылки на методы

Функциональные интерфейсы позволяют передавать **поведение** как параметр метода.

Например:

```
List<String> words = Arrays.asList("apple", "banana");
```

```
words.forEach(System.out::println); // Consumer<String>
```

2. Компактность и читаемость

Использование функциональных интерфейсов делает код более лаконичным и понятным, особенно при работе с потоками данных и коллекциями.

3. Совместимость с Stream API

`Predicate`, `Function` и `BinaryOperator` активно применяются в методах `filter()`, `map()` и `reduce()`, что позволяет строить цепочки обработки данных.

40. Функциональные интерфейсы: Consumer, Supplier, UnaryOperator.

Функциональные интерфейсы в Java

Функциональный интерфейс — это интерфейс с **одним абстрактным методом**, предназначенный для использования с **лямбда-выражениями**, ссылками на методы и концепциями функционального программирования. Он позволяет **передавать поведение как параметр**, что делает код гибким, компактным и читаемым.

Java предоставляет стандартные функциональные интерфейсы в пакете `java.util.function`, которые покрывают большинство типичных операций с данными.

Основные интерфейсы

1. Consumer<T>

`Consumer` представляет собой функцию, которая **принимает объект типа T и ничего не возвращает**. Основной метод — `void accept(T t)`.

Применение: выполнение действия над объектом, например, вывод на консоль, логирование или обновление состояния объекта.

Пример:

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.forEach(name -> System.out.println(name)); //
Consumer<String>
```

2. Supplier<T>

`Supplier` представляет **источник объектов типа T, который не принимает аргументов, но возвращает результат**. Основной метод — `T get()`.

Применение: генерация объектов, ленивые вычисления, получение значений по требованию.

Пример:

```
Supplier<Double> randomSupplier = () -> Math.random();
System.out.println(randomSupplier.get()); // возвращает случайное
число
```

3. UnaryOperator<T>

`UnaryOperator` представляет **операцию над объектом типа T, возвращающую результат того же типа**. Он наследует `Function<T, T>`, а основной метод — `T apply(T t)`.

Применение: преобразование или модификация объектов одного типа, например, увеличение числового значения или изменение строки.

Пример:

```
UnaryOperator<Integer> square = x -> x * x;
System.out.println(square.apply(5)); // 25
```

Применение функциональных интерфейсов

Функциональные интерфейсы активно используются в **Stream API**:

- **Consumer** — для `forEach()`;
- **Supplier** — для `generate()`;
- **UnaryOperator** — для `map()`.

Они позволяют создавать универсальный код, где поведение методов можно передавать как параметр, делая код компактным, гибким и удобным для повторного использования.

41. Понятие многопоточности.

Многопоточность — это способность программы выполнять **несколько потоков исполнения (threads) одновременно**. Поток представляет собой независимую последовательность выполнения кода, которая может работать параллельно с другими потоками внутри одного процесса. Это позволяет разделять задачи внутри программы, повышая её производительность и эффективность.

В Java **поток (Thread)** — это базовая единица выполнения, а **процесс** — это программа с собственным адресным пространством. Многопоточность позволяет одному процессу содержать несколько потоков, которые могут выполняться либо **параллельно** на многопроцессорных системах, либо **поочередно** через планировщик CPU, создавая эффект многозадачности

Основные характеристики потоков

- Каждый поток имеет своё **состояние**, идентификатор и контекст выполнения.
- Потоки могут **делить ресурсы процесса**, такие как память и объекты, что позволяет эффективно обмениваться данными между задачами.
- При работе с потоками важно учитывать **синхронизацию**, чтобы избежать конфликтов при одновременном доступе к общим данным.

Применение многопоточности

Многопоточность широко используется для:

- Одновременного выполнения нескольких задач, например, обработка пользовательского ввода, выполнение вычислений и вывод данных.
- Оптимизации ресурсоёмких операций, таких как загрузка файлов, сетевое взаимодействие или работа с базой данных.
- Реализации серверов и приложений с высокой нагрузкой, где одновременно обслуживаются несколько клиентов.
- Повышения производительности программ на многопроцессорных системах, эффективно используя все доступные ядра.

Пример

В Java поток можно создать двумя способами:

1. Наследуя класс Thread и переопределяя метод run() :

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Поток выполняется: " +  
Thread.currentThread().getName());  
    }  
}  
new MyThread().start();
```

2. Реализуя интерфейс Runnable:

```
Runnable task = () -> System.out.println("Поток через Runnable");  
new Thread(task).start();
```

Оба способа создают новый поток, который выполняется параллельно с основным потоком программы.

42. Создание и выполнение потоков в Java.

В Java существует два основных способа создания потоков исполнения: **наследование от класса Thread** и **реализация интерфейса Runnable**.

1. Наследование от Thread

Создается собственный класс, который наследует Thread, и переопределяется метод `run()`, содержащий код потока. Для запуска нового потока вызывается метод `start()`. Важно понимать, что метод **`start()` создает новый поток и вызывает `run()` в нем**, а прямой вызов `run()` просто выполнит код в текущем потоке, не создавая нового.

Пример:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Поток выполняется: " +
Thread.currentThread().getName());
    }
}
```

```
new MyThread().start();
```

2. Реализация интерфейса Runnable

Создается класс, реализующий Runnable, и определяется метод `run()`. Новый поток создается через объект Thread, которому передается экземпляр Runnable, и запускается методом `start()`. Этот способ позволяет использовать один объект Runnable в нескольких потоках и удобен для работы с наследованием других классов.

Пример:

```
Runnable task = () -> System.out.println("Поток через Runnable");
new Thread(task).start();
```

Жизненный цикл потока

Каждый поток в Java проходит через несколько состояний:

1. **Создан (New)** — объект потока создан, но `start()` ещё не вызван.
2. **Запущен (Runnable)** — поток готов к выполнению и ожидает планировщика CPU.
3. **Выполняется (Running)** — поток выполняет метод `run()`.
4. **Ожидание (Waiting / Timed Waiting / Blocked)** — поток приостановлен до наступления определенного события (например, освобождения ресурса или завершения другого потока).
5. **Завершён (Terminated)** — выполнение метода `run()` окончено. Поток больше не может быть перезапущен.

Основные методы класса Thread

- `start()` — запуск нового потока.
- `run()` — содержит код, который будет выполняться в потоке.
- `sleep(long millis)` — приостановка выполнения потока на указанное время.
- `join()` — ожидание завершения другого потока перед продолжением текущего.
- `setPriority(int)` — установка приоритета потока (от 1 до 10).
- `isAlive()` — проверка, выполняется ли поток в данный момент.

43. Класс Thread.

Класс Thread в Java

В Java **Thread** — это встроенный класс, предназначенный для создания и управления **потоками выполнения**. Каждый объект класса Thread представляет отдельный поток, который может работать параллельно с другими потоками в рамках одного процесса. Потоки позволяют одновременно выполнять несколько задач, повышая производительность и отзывчивость приложений.

Основные возможности класса Thread

1. Создание и запуск потоков

Переопределение метода `run()` определяет код, который будет выполняться в потоке. Метод `start()` запускает поток в отдельном потоке исполнения, вызывая `run()`.

Пример:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Поток выполняется: " +
Thread.currentThread().getName()); }
new MyThread().start();
```

2. Управление потоком

`sleep(long millis)` — приостанавливает выполнение потока на заданное время.

`join()` — ожидает завершения другого потока перед продолжением текущего.

`setPriority(int)` — устанавливает приоритет потока (от 1 до 10).

`interrupt()` — прерывает поток, позволяя обработать прерывание.

Пример с `join()`:

```
Thread t1 = new Thread(() -> System.out.println("Поток 1"));
Thread t2 = new Thread(() -> System.out.println("Поток 2"));
t1.start();
t1.join(); // основной поток ждёт завершения t1
t2.start();
```

3. Получение информации о потоке

- `getName()` и `setName(String)` — получение и установка имени потока.
- `getPriority()` — получение приоритета потока.
- `isAlive()` — проверка, активен ли поток в данный момент.
- `currentThread()` — получение объекта текущего выполняющегося потока.

Способы использования класса Thread

1. Наследование от Thread

Создается подкласс Thread с переопределением метода `run()`. Запуск осуществляется через `start()`. Этот способ удобен для простых потоков, но ограничивает наследование, так как Java не поддерживает множественное наследование классов.

2. Использование с интерфейсом Runnable

Класс реализует интерфейс Runnable, метод `run()` содержит код потока. Объект Runnable передается в конструктор Thread, и поток запускается методом `start()`. Этот способ более гибкий, так как позволяет использовать наследование других классов и повторно применять один объект Runnable в нескольких потоках.

Пример:

```
Runnable task = () -> System.out.println("Поток через Runnable");
Thread thread = new Thread(task);
thread.start();
```

44. Интерфейс Runnable.

Интерфейс Runnable в Java

Runnable — это функциональный интерфейс, предназначенный для создания потоков выполнения **без наследования от класса Thread**. Он позволяет **разделять логику потока и управление потоком**, что делает код более гибким и повторно используемым.

Основные особенности

- Содержит **один абстрактный метод**: `void run();`

В этом методе реализуется код, который будет выполняться в потоке.

- Объект Runnable можно передать в конструктор Thread, а запуск потока осуществляется через `start()`.
- Использование Runnable позволяет реализовать многопоточность **без наследования Thread**, что важно, если класс уже наследует другой класс (в Java поддерживается только одиночное наследование).

Применение Runnable

1. Создание потока через Runnable

- Класс реализует интерфейс Runnable.
- Метод `run()` содержит код потока.
- Создается объект Thread с этим Runnable и запускается методом `start()`.

Пример:

```
class MyTask implements Runnable {  
    public void run() {  
        System.out.println("Поток выполняется: " +  
Thread.currentThread().getName());  
    }  
}
```

```
Thread t = new Thread(new MyTask());  
t.start();
```

2. Разделение логики и управления

- Код потока полностью отделен от класса Thread.
- Один и тот же объект Runnable может использоваться для запуска

нескольких потоков:

```
Runnable task = () -> System.out.println("Поток работает");  
new Thread(task).start();  
new Thread(task).start();
```

3. Использование с лямбда-выражениями

Так как Runnable — функциональный интерфейс, его удобно использовать с лямбда-выражениями:

```
Runnable task = () -> System.out.println("Поток через лямбду");  
new Thread(task).start();
```

45. Синхронизация потоков. Оператор `synchronized`.

Синхронизация потоков — это механизм, который предотвращает **одновременный доступ нескольких потоков к общим ресурсам**, таким как поля объектов или разделяемые данные.

Без синхронизации возможны **конфликты, потеря данных или некорректные результаты** из-за одновременного изменения одного ресурса.

Оператор `synchronized`

`synchronized` — ключевое слово Java, которое используется для **блокировки критической секции кода**, чтобы в каждый момент времени доступ к ней имел только один поток.

Применение:

1. Синхронизация методов

- Объектные методы: блокировка привязывается к конкретному объекту.

```
public synchronized void increment() {  
    count++;  
}
```

- Статические методы: блокировка привязывается к классу, общий монитор для всех экземпляров.

2. Синхронизация блоков кода

Можно блокировать только определённую часть метода для повышения производительности.

```
synchronized(this) {  
    count++;  
}
```

Принцип работы:

Поток, вошедший в `synchronized`-блок, занимает **монитор объекта**, и другие потоки должны ждать, пока монитор не будет освобожден. Это обеспечивает **эксклюзивный доступ** к ресурсу.

Применение синхронизации

- Защита **общих ресурсов**, например счетчиков, коллекций или файлов;
- Предотвращение **гонок потоков (race conditions)**;
- Обеспечение **консистентности данных** при многопоточном доступе;
- Используется в сочетании с методами `wait()`, `notify()` и `notifyAll()` для организации координации потоков.

46. Синхронизация потоков. Класс Phaser.

Класс `Phaser` в Java

Понятие

Phaser — это инструмент синхронизации потоков, позволяющий координировать выполнение нескольких потоков **по фазам**. Он является более гибкой альтернативой `CyclicBarrier` и `CountDownLatch`, так как поддерживает многофазные задачи и динамическое добавление или удаление потоков.

Основные возможности Phaser

1. Многофазная синхронизация

Потоки могут работать по шагам (фазам). Каждый поток должен дождаться всех остальных, прежде чем перейти к следующей фазе, что гарантирует согласованность выполнения.

2. Регистрация потоков

Потоки регистрируются в объекте `Phaser` с помощью метода `register()`. После завершения своей фазы поток вызывает `arriveAndAwaitAdvance()`, чтобы сигнализировать о готовности и ожидать остальных потоков.

3. Методы класса Phaser

- `register()` — зарегистрировать новый поток.
- `arrive()` — уведомить `Phaser` о завершении фазы, **без ожидания** других потоков.
- `arriveAndAwaitAdvance()` — уведомить о завершении фазы и дождаться остальных потоков.
- `getPhase()` — получить номер текущей фазы.
- `bulkRegister(int parties)` — зарегистрировать несколько потоков одновременно.
- `arriveAndDeregister()` — завершить участие потока и удалить его из координации.

Применение

- Координация потоков в **многофазных задачах**, например:
 - вычисления по шагам,
 - обработка данных в нескольких этапах,
 - сложные симуляции или игры с фазовой логикой;
- Организация синхронизации **динамически изменяющегося числа потоков**;
- Более гибкая альтернатива `CyclicBarrier` для многопоточных приложений.

47. Синхронизация потоков. Блокировки. ReentrantLock

Блокировки и класс `ReentrantLock` в Java

Понятие

Блокировка (Lock) — это механизм синхронизации потоков, который позволяет **управлять доступом к общим ресурсам**.

Класс **`ReentrantLock`** из пакета `java.util.concurrent.locks` является гибкой альтернативой оператору `synchronized`.

- **Reentrant** означает, что поток, уже владеющий блокировкой, может захватывать её повторно без блокировки самого себя.

Основные возможности `ReentrantLock`

1. Создание блокировки

```
2. ReentrantLock lock = new ReentrantLock();
```

3. Захват и освобождение блокировки

```
4. lock.lock();    // захват блокировки
```

```
5. try {
```

```
6.     // критическая секция
```

```
7. } finally {
```

```
8.     lock.unlock(); // освобождение блокировки
```

```
9. }
```

10. Дополнительные возможности

- **`tryLock()`** — попытка захватить блокировку без ожидания;
- **`lockInterruptibly()`** — позволяет прерывать поток, ожидающий блокировку;
- **`getHoldCount()`** — количество повторных захватов блокировки текущим потоком;
- **`isLocked()`** — проверка, занята ли блокировка.

Преимущества перед `synchronized`

- Более **гибкий контроль** над захватом и освобождением блокировки;
- Возможность **прерывания потоков**, ожидающих блокировку;
- Возможность **попытки захвата блокировки без ожидания**;
- Поддержка **повторного захвата блокировки одним потоком**.

Применение

- Защита **разделяемых ресурсов** при многопоточном доступе;
- Реализация **сложной логики синхронизации**, где `synchronized` недостаточно;
- Создание **параллельных алгоритмов и многопоточных серверов** с безопасным доступом к данным.

48. Методы `wait()`, `notify()`, `notifyAll()`.

Эти методы используются для **координации работы потоков**, которые обращаются к **общим ресурсам**.

Они позволяют одному потоку **ожидать события**, пока другой поток **сигнализирует о готовности данных или завершении работы**.

-- Все три метода принадлежат классу `Object` и могут быть вызваны **только из синхронизированного блока** (`synchronized`).

Методы

1. `wait()`

- Приостанавливает выполнение текущего потока и **освобождает монитор объекта**;
- Поток будет ожидать **сигнала от другого потока** (`notify()` или `notifyAll()`), чтобы продолжить выполнение.
- Существует перегрузка с таймаутом: `wait(long millis)` — поток ждёт указанное время.

2. `notify()`

- Пробуждает **один поток**, ожидающий монитор объекта;
- Поток выбирается планировщиком JVM (не обязательно тот, который первый вызвал `wait()`).

3. `notifyAll()`

- Пробуждает **все потоки**, ожидающие монитор объекта;
- Используется, когда несколько потоков могут продолжить работу после изменения состояния ресурса.

Пример использования

```
synchronized(sharedResource) {  
    while (!condition) {  
        sharedResource.wait(); // поток ждёт  
    }  
    // работа с ресурсом  
    sharedResource.notify(); // пробуждает другой поток  
}
```

- Поток ждёт до тех пор, пока **не изменится состояние ресурса**;
- `notify()` или `notifyAll()` сигнализируют другим потокам, что **ресурс готов к использованию**.

49. Семафоры.

Семафоры (Semaphore) в Java

Понятие

Семафор — это механизм синхронизации потоков, который **контролирует количество потоков**, одновременно получающих доступ к ресурсу.

Он используется для управления доступом к **ограниченным ресурсам**, например, сетевым соединениям, файловым системам или базе данных.

Класс Semaphore

- Находится в пакете `java.util.concurrent`.
- Инициализируется числом **разрешений (permits)** — количеством потоков, которые могут одновременно работать с ресурсом.
- Основные методы:
 - **acquire()** — захват разрешения, поток блокируется, если оно недоступно;
 - **release()** — освобождение разрешения, позволяющее другому потоку продолжить работу;
 - **availablePermits()** — получение текущего количества доступных разрешений;
 - **tryAcquire()** — попытка захвата разрешения без ожидания.

Применение

- Ограничение **одновременного доступа к ресурсу** для нескольких потоков;
- Организация **пула потоков** и управление их работой;
- Реализация схем **producer-consumer**;
- Контроль **критических секций кода**, где ресурсы ограничены.

Преимущества

- Позволяет избежать **конфликтов при многопоточном доступе**;
- Обеспечивает **гибкую и безопасную синхронизацию**;
- Подходит для **динамического числа потоков**, так как количество разрешений можно изменять в процессе работы.

50. Обмен между потоками. Класс `Exchanger`.

Класс `Exchanger` в Java

Понятие

`Exchanger` — это инструмент синхронизации потоков, позволяющий двум потокам обмениваться данными.

Каждый поток передает объект другому потоку и получает от него свой объект, после чего оба могут продолжить работу.

- Используется в **многопоточных приложениях**, где потоки должны **согласовывать промежуточные результаты**.

Основные особенности

- Находится в пакете `java.util.concurrent`.
- Каждый `Exchanger` предназначен для **обмена между двумя потоками**;
- Основной метод:
 - **`exchange(V x)`** — передает объект `x` другому потоку и **ожидает объект от него**;
- Если второй поток еще не вызвал `exchange()`, первый поток **ждет**;
- Поддерживает **временные ограничения** через перегрузку с таймаутом.

Применение

- Согласование потоков при **пошаговой обработке данных**;
- Обмен результатами **параллельных вычислений**;
- Реализация схем типа **`producer-consumer`**, где данные должны быть переданы точно между потоками;
- Создание **конвейеров обработки данных**, где потоки работают параллельно, но обмениваются промежуточными результатами.

Преимущества

- Обеспечивает **безопасный и простой обмен данными между двумя потоками**;
- Исключает необходимость использования **дополнительных буферов или сложных блокировок**;
- Подходит для **координации фазовых операций**, где результаты одного потока нужны другому.

51. Основы Stream API.

Stream API в Java

Понятие

Stream API — это набор инструментов для **обработки последовательностей данных** в функциональном стиле.

Стримы **не хранят данные**, а **операции выполняются над источником данных**, например, коллекциями, массивами или файлами.

- Stream API появился в Java 8 и позволяет писать **короткий, читаемый и параллельный код**.

Основные особенности

- **Потоковый подход**: данные обрабатываются **пошагово через цепочку операций**.
- **Ленивое выполнение**: промежуточные операции выполняются **только при терминальной операции**.
- **Функциональный стиль**: использование лямбд и функциональных интерфейсов (Predicate, Function, Consumer).
- Возможность **параллельной обработки** через `parallelStream()`.

Типы операций

1. **Промежуточные (intermediate)**
 - Возвращают новый Stream, операции **ленивые**;
 - Примеры: `filter()`, `map()`, `sorted()`, `distinct()`.
2. **Терминальные (terminal)**
 - Завершают обработку и возвращают **результат**;
 - Примеры: `collect()`, `forEach()`, `count()`, `reduce()`.

Применение

- Обработка коллекций: фильтрация, преобразование, сортировка, агрегация;
- Упрощение **логики обработки данных** без явных циклов и условий;
- Параллельная обработка больших объемов данных для повышения **производительности**;
- Легкая интеграция с **функциональными интерфейсами и лямбда-выражениями**.

Преимущества Stream API

- **Читаемость и лаконичность кода**;
- Возможность **цепочек операций** без создания промежуточных коллекций;
- Поддержка **параллельного выполнения**;
- Полная совместимость с **коллекциями и массивами Java**.

52. Stream API. Создание потока данных

Stream API. Создание потока данных

Понятие

Stream — это **последовательность элементов**, над которыми можно выполнять **функциональные операции** (фильтрация, преобразование, агрегация).

Потоки **не хранят данные**, а обрабатывают их из источника — коллекций, массивов, файлов или генераторов.

Основные источники потоков

1. Коллекции (Collection)

- `stream()` — последовательный поток;
- `parallelStream()` — параллельный поток.

2. Массивы

- Используется класс `Arrays` и метод `Arrays.stream(array)`.

3. Файлы и I/O

- `Files.lines(Path)` — поток строк из файла.

4. Статические методы Stream

- `Stream.of(...)` — создание потока из заданных элементов;
- `Stream.generate()` — генерация бесконечного потока;
- `Stream.iterate()` — создание потока по правилу итерации.

Особенности создания потоков

- Поток **не хранит данные**, а лишь предоставляет операции над источником;
- Поток **ленивый** — промежуточные операции выполняются только при терминальной операции;
- Потоки можно **создавать многократно**, они не могут использоваться повторно после терминальной операции.

Применение

- Обработка коллекций и массивов в функциональном стиле;
- Создание **конвейеров обработки данных** (`map`, `filter`, `sorted`, `reduce`);
- Параллельная обработка данных для повышения производительности;
- Генерация **динамических или бесконечных потоков** для сложных вычислений.

1. Примеры создания потоков из разных источников

// Из коллекции

```
List<String> list = List.of("a", "b", "c");
```

```
Stream<String> stream1 = list.stream(); // последовательный поток
```

```
Stream<String> stream2 = list.parallelStream(); // параллельный  
поток
```

// Из массива

```
int[] numbers = {1, 2, 3};
```

```
IntStream stream3 = Arrays.stream(numbers);
```

// Из файла

```
Path path = Paths.get("file.txt");
```

```
Stream<String> stream4 = Files.lines(path);
```

// Через статические методы Stream

```
Stream<String> stream5 = Stream.of("x", "y", "z");
```

```
Stream<Double> randomStream = Stream.generate(Math::random); //
бесконечный поток
Stream<Integer> iterStream = Stream.iterate(1, n -> n + 1); //
последовательность 1,2,3...
```

2. Особенности потоков

- Потоки **ленивые** — промежуточные операции (`map`, `filter`) выполняются только при **терминальной операции** (`collect`, `forEach`, `reduce`).
- После терминальной операции поток **нельзя использовать повторно**.

3. Применение на практике

- Фильтрация и преобразование коллекций: `list.stream().filter(s -> s.startsWith("a")).map(String::toUpperCase)`
- Параллельная обработка больших массивов или списков: `list.parallelStream().map(...).reduce(...)`
- Создание **конвейеров обработки данных** для чистого функционального стиля.

53.Stream API. Метод collect.

Stream API. Метод `collect()`

Понятие

`collect()` — это терминальная операция **Stream API**, которая позволяет преобразовать поток данных в коллекцию или другой контейнер.

Она используется для сбора результатов обработки после цепочки промежуточных операций (`map`, `filter`, `sorted` и т.д.).

Основные особенности

- Метод **`collect()`** завершает работу потока и возвращает объект любого типа, обычно коллекцию (`List`, `Set`, `Map`) или агрегированный результат;
- Используется класс **`Collectors`** для стандартных операций сбора данных;
- Позволяет выполнять:
 - Преобразование потока в список или множество (`Collectors.toList()`, `Collectors.toSet()`);
 - Группировку по ключу (`Collectors.groupingBy()`);
 - Объединение строк (`Collectors.joining()`);
 - Суммирование или агрегацию чисел (`Collectors.summingInt()`, `averagingDouble()`).

Применение

- Преобразование потоков в коллекции для дальнейшего использования;
- Агрегация данных после обработки коллекций;
- Удобный инструмент для **фильтрации, группировки и статистических операций** в функциональном стиле;
- Используется как **терминальная операция**, завершающая цепочку **Stream API**.

Преимущества

- Позволяет писать **короткий, читаемый код** без ручного цикла и добавления элементов;
- Совместим с различными коллекциями и структурами данных;
- Поддерживает **многоуровневые агрегирующие операции** и группировки.

54. Stream API. Тип Optional.

Optional — это контейнерный объект, который может содержать значение или быть пустым.

Он используется для **безопасной работы с потенциально отсутствующими данными**, чтобы избежать `NullPointerException`.

- Широко применяется в **Stream API**, где методы могут возвращать **один элемент или результат агрегации**, который может отсутствовать.

Основные особенности

- `Optional` **не является коллекцией**, а объектом-обёрткой для одного значения;
- Основные методы:

`isPresent()` — проверяет, содержит ли объект значение;

`get()` — возвращает значение (если оно есть);

`orElse(T other)` — возвращает значение или альтернативу, если пусто;

`orElseGet(Supplier<? extends T> other)` — ленивое вычисление альтернативы;

`ifPresent(Consumer<? super T> action)` — выполняет действие, если значение присутствует.

- Используется для безопасной работы с методами **Stream API**, которые могут вернуть пустой результат:
 - `findFirst()`, `findAny()`, `max()`, `min()`.

Применение

- Получение результата из потока, который может быть пустым;
- Предотвращение **NullPointerException** при работе с одиночными элементами;
- Лаконичное использование с **лямбда-выражениями** и функциональными интерфейсами;
- Удобно для **агрегационных и поисковых операций** в потоках данных.

Преимущества

- Безопасная работа с **необязательными значениями**;
- Упрощает чтение и обработку данных без явных проверок на `null`;
- Поддерживает **функциональный стиль программирования** в Java 8+.

55. Stream API. Получение подпотока и объединение потоков.

Stream API. Получение подпотока и объединение потоков

Понятие

В **Stream API** потоки данных можно **разделять на подпотоки** или **объединять несколько потоков** для удобной обработки данных. Это позволяет **выбирать часть данных** или **комбинировать результаты** из разных источников.

Получение подпотока

- **Подпоток** создаётся с помощью промежуточных операций, которые **фильтруют или ограничивают поток**:
 - `filter(Predicate)` — выбирает элементы по условию;
 - `limit(long n)` — оставляет только первые `n` элементов;
 - `skip(long n)` — пропускает первые `n` элементов;
 - `distinct()` — исключает дубликаты.
- Подпоток **не копирует данные**, а создаёт **новую цепочку операций** над исходным источником.

Объединение потоков

- Потоки можно объединять с помощью метода **`Stream.concat()`**:
 - Создаёт единый поток, который содержит элементы **двух исходных потоков**;
- Позволяет комбинировать данные из **разных коллекций или массивов**;
- После объединения можно применять любые **промежуточные и терминальные операции**, как для обычного потока.

Применение

- Получение **части данных для обработки**;
- Фильтрация или выборка элементов из больших коллекций;
- Объединение данных из нескольких источников в **один поток** для дальнейшей обработки;
- Используется для построения **параллельных конвейеров данных**.

56. Понятие паттерна проектирования. Классификация паттернов.

Понятие

Паттерн проектирования (Design Pattern) — это готовое, проверенное решение типичной задачи проектирования, которое можно повторно использовать в различных приложениях.

- Паттерн **не является кодом, а шаблоном**, описывающим взаимодействие классов, объектов и их обязанности;
- Основная цель — **улучшение структуры кода, гибкости и повторного использования**.

Основные характеристики

- Решает **типичные проблемы проектирования**, которые часто встречаются в программировании;
- Снижает **сложность и дублирование кода**;
- Упрощает **поддержку и расширение программного обеспечения**;
- Повышает **читаемость и понимание архитектуры** для команды разработчиков.

Классификация паттернов

Паттерны проектирования делятся на три большие группы:

1. Порождающие (Creational)

- Отвечают за **создание объектов**;
- Позволяют **управлять процессом инстанцирования** и скрывать детали реализации;
- Примеры: Singleton, Factory Method, Abstract Factory, Builder, Prototype.

2. Структурные (Structural)

- Определяют **структуру классов и объектов**, показывая, как они взаимосвязаны;
- Обеспечивают **удобное объединение объектов и повторное использование компонентов**;
- Примеры: Adapter, Decorator, Facade, Composite, Proxy, Bridge.

3. Поведенческие (Behavioral)

- Определяют **поведение объектов и их взаимодействие**;
- Позволяют **организовать обмен данными и выполнение операций между объектами**;
- Примеры: Observer, Strategy, Command, Template Method, Iterator, State, Mediator.

68. Порождающий паттерн «Фабричный метод».

Порождающий паттерн «Фабричный метод» (Factory Method)

Понятие

Фабричный метод — это порождающий паттерн проектирования, который **определяет интерфейс для создания объектов**, но позволяет **подклассам решать, какой конкретный объект создавать**.

- Основная цель: **отделить код, который использует объект, от кода, который его создает**;
- Позволяет **добавлять новые типы объектов без изменения клиентского кода**.

Основные характеристики

- Имеется **интерфейс или абстрактный класс с методом создания объектов**;
- Подклассы реализуют этот метод и создают **конкретные объекты**;
- Клиент работает с **интерфейсом или базовым классом**, не зная конкретной реализации;
- Упрощает **расширение и поддержку** без модификации существующего кода.

Применение

- Создание **объектов с разными реализациями**, которые разделяют общий интерфейс;
- Когда **точный тип объекта известен только в момент выполнения**;
- В **фреймворках и библиотеках**, где пользователю нужно предоставить объект без знания его конкретного класса;
- Используется для **создания разнообразных продуктов по единообразному интерфейсу**.

Преимущества

- Разделяет **создание и использование объектов**;
- Облегчает **расширение системы** без изменения существующего кода;
- Повышает **гибкость и повторное использование кода**;
- Снижает **зависимость клиентского кода от конкретных классов**.

69.Порождающий паттерн «Абстрактная фабрика».

Абстрактная фабрика — это порождающий паттерн проектирования, который позволяет создавать семейства взаимосвязанных объектов без указания их конкретных классов.

- Основная цель: **обеспечить единый интерфейс для создания объектов разных типов**, которые работают вместе;
- Часто используется, когда система должна поддерживать **несколько семейств продуктов** с одинаковым интерфейсом.

Основные характеристики

- Существует **абстрактная фабрика** с методами для создания объектов разных семейств;
- **Конкретные фабрики** реализуют эти методы для создания конкретных объектов;
- Клиент работает **через интерфейс абстрактной фабрики**, не зная конкретных классов;
- Позволяет **легко менять семейство продуктов**, не изменяя клиентский код.

Применение

- Создание **совместимых объектов из одного семейства** (например, GUI-компоненты для разных платформ);
- Поддержка **плагинов и расширяемых систем**, где объекты должны работать согласованно;
- Когда нужно **разделить создание объектов и их использование**;
- В системах с **множеством семейств объектов**, требующих единообразного интерфейса.

Преимущества

- Обеспечивает **согласованность объектов** внутри одного семейства;
- Упрощает **расширение и поддержку системы**;
- Позволяет **заменять семейство объектов**, не меняя клиентский код;
- Повышает **гибкость и модульность архитектуры**.

Итог

Абстрактная фабрика:

- предоставляет **интерфейс для создания семейств взаимосвязанных объектов**;
- позволяет клиенту **работать с объектами через абстракцию**, не зная конкретных реализаций;
- упрощает **масштабирование, поддержку и модульное расширение системы**;
- широко применяется в **кроссплатформенных и расширяемых приложениях**.

70.Порождающий паттерн «Строитель».

Порождающий паттерн «Строитель» (Builder)

Понятие

Строитель — это порождающий паттерн проектирования, который **отделяет процесс построения сложного объекта от его представления**, позволяя создавать разные варианты объекта пошагово.

- Основная цель: **упростить создание сложных объектов с множеством параметров;**
- Позволяет **конструировать объект по частям, сохраняя контроль над процессом сборки.**

Основные характеристики

- Состоит из **Director (директор)**, который управляет процессом сборки;
- **Builder (строитель)** предоставляет интерфейс для пошагового создания частей объекта;
- **ConcreteBuilder (конкретный строитель)** реализует конкретные шаги сборки;
- Клиент получает **готовый объект**, не заботясь о деталях сборки.

Применение

- Создание **объектов с множеством параметров или конфигураций;**
- Формирование **различных представлений одного объекта** (например, текстовые и графические отчеты);
- В системах, где **необходимо пошаговое построение объектов;**
- Часто используется вместе с **порождающими паттернами** для упрощения процесса создания.

Преимущества

- Разделяет **создание объекта и его представление;**
- Позволяет **пошагово строить сложные объекты;**
- Обеспечивает **повторное использование кода и модульность;**
- Упрощает **расширение и изменение процесса сборки**, не меняя клиентский код.

71.Порождающий паттерн «Прототип».

Порождающий паттерн «Прототип» (Prototype)

Понятие

Прототип — это порождающий паттерн проектирования, который позволяет создавать новые объекты копированием существующих экземпляров (клонированием).

- Основная цель: **обеспечить быстрый и удобный способ создания объектов, особенно когда их создание с нуля сложно или дорого;**
- Позволяет динамически создавать объекты без привязки к конкретным классам.

Основные характеристики

- Класс реализует интерфейс **Cloneable** и метод `clone()` для копирования объекта;
- **Клонирование может быть глубоким (deep copy) или поверхностным (shallow copy);**
- Клиент получает копию объекта, не создавая его напрямую через конструктор;
- Используется для динамического создания объектов во время выполнения.

Применение

- Создание **сложных объектов с множеством параметров**, которые сложно инициализировать заново;
- **Кэширование объектов** и повторное использование экземпляров;
- В системах, где требуется динамическое создание объектов без знания их конкретного класса;
- Применяется для игровых объектов, графических элементов и документов.

Преимущества

- Позволяет **создавать объекты быстро и гибко;**
- Упрощает добавление новых типов объектов без изменения клиентского кода;
- Снижает зависимость от конкретных классов;
- Удобен для создания прототипов и шаблонных объектов.

72.Порождающий паттерн «Одиночка».

Порождающий паттерн «Одиночка» (Singleton)

Понятие

Одиночка (Singleton) — это порождающий паттерн проектирования, который **ограничивает создание класса одним экземпляром** и предоставляет **глобальную точку доступа** к этому экземпляру.

- Основная цель: **гарантировать, что существует только один объект класса**, и предоставить удобный способ к нему обращаться;
- Часто используется для **централизованного управления ресурсами**.

Основные характеристики

- Конструктор класса **закрит (private)**, чтобы предотвратить создание объектов извне;
- Существует **статический метод** (`getInstance()`), который возвращает единственный экземпляр;
- Экземпляр создаётся **лениво (по необходимости)** или **при загрузке класса**;
- Может быть **потокобезопасным** при использовании многопоточности.

Применение

- Управление **глобальными ресурсами**: базы данных, логгеры, конфигурации;
- **Настройки приложения**, которые должны быть общими для всех компонентов;
- В **системах кэширования** или для управления пулом соединений;
- Когда важно **гарантировать уникальность объекта в системе**.

Преимущества

- Гарантирует **единственный экземпляр класса**;
- Обеспечивает **глобальный доступ к объекту**;
- Упрощает управление **глобальными ресурсами**;
- Повышает **контроль над жизненным циклом объекта**.

Есть несколько способов реализовать Singleton, я покажу **классический потокобезопасный вариант с ленивой инициализацией**:

```
// Singleton.java
public class Singleton {

    // Приватное статическое поле для хранения единственного
    // экземпляра
    private static volatile Singleton instance;

    // Приватный конструктор, чтобы никто не мог создать объект
    // извне
    private Singleton() {
        System.out.println("Singleton создан!");
    }

    // Публичный метод для получения экземпляра
    public static Singleton getInstance() {
        if (instance == null) { // Проверка без
            // синхронизации
            synchronized (Singleton.class) { // Блокировка для
                // потокобезопасности
                if (instance == null) { // Проверка снова
                    // внутри блока
```

```

        instance = new Singleton();
    }
}
return instance;
}

// Пример метода
public void doSomething() {
    System.out.println("Метод Singleton вызван");
}
}

```

Пример использования:

```

public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        s1.doSomething();

        System.out.println(s1 == s2); // true, оба ссылаются на
    }
}

```

один экземпляр

1. Конструктор приватный — нельзя создать объект через new.
2. Метод `getInstance()` возвращает **один и тот же экземпляр**.
3. `volatile` + двойная проверка (`if (instance == null)`) обеспечивает **потокобезопасность** и минимальное количество блокировок.

73. Структурный паттерн «Адаптер».

Структурный паттерн «Адаптер» (Adapter)

Понятие

Адаптер — это структурный паттерн проектирования, который позволяет **совместить несовместимые интерфейсы**, чтобы объекты разных классов могли работать вместе.

- Основная цель: **предоставить клиенту единый интерфейс для работы с объектами, которые иначе были бы несовместимы;**
- Часто используется для **интеграции старого и нового кода.**

Основные характеристики

- Существует **целевой интерфейс (Target)**, который ожидает клиент;
- **Адаптируемый класс (Adaptee)** содержит функциональность, которую нужно использовать;
- **Адаптер (Adapter)** реализует Target и внутри вызывает методы Adaptee;
- Клиент **работает через Target**, не зная о деталях Adaptee.

Применение

- Интеграция **старых библиотек или классов с новым кодом;**
- Объединение **разных интерфейсов без изменения исходных классов;**
- Преобразование данных между **несовместимыми структурами или форматами;**
- Используется в **GUI, системах ввода-вывода и API-интеграциях.**

Преимущества

- Позволяет **повторно использовать существующие классы;**
- Снижает **зависимость клиента от конкретных реализаций;**
- Упрощает **интеграцию и расширение системы;**
- Поддерживает **гибкость архитектуры и модульность кода.**

74. Структурный паттерн «Мост».

Структурный паттерн «Мост» (Bridge)

Понятие

Мост — это структурный паттерн проектирования, который **разделяет абстракцию и её реализацию**, позволяя изменять их **независимо друг от друга**.

- Основная цель: **снизить связность между абстракцией и реализацией**;
- Позволяет создавать **гибкую и расширяемую архитектуру**, где абстракции и конкретные реализации можно комбинировать произвольно.

Основные характеристики

- Существует **Абстракция (Abstraction)**, которая определяет интерфейс для клиента;
- **Реализация (Implementor)** предоставляет конкретные методы работы;
- Абстракция хранит ссылку на реализацию и **делегирует ей выполнение операций**;
- Позволяет **добавлять новые абстракции и реализации независимо**.

Применение

- Разделение **интерфейса и реализации** в GUI-компонентах;
- Создание систем с **множеством платформ и вариантов реализации**;
- Когда требуется **изменять абстракцию и реализацию независимо**, например, разные способы сохранения данных;
- Используется в **графических фреймворках, системах ввода-вывода и кроссплатформенных приложениях**.

Преимущества

- Обеспечивает **слабую связность между абстракцией и реализацией**;
- Позволяет **расширять абстракцию и реализацию независимо**;
- Упрощает **поддержку и масштабирование системы**;
- Повышает **гибкость и повторное использование кода**.

75. Структурный паттерн «Компоновщик».

Структурный паттерн «Компоновщик» (Composite)

Понятие

Компоновщик — это структурный паттерн проектирования, который позволяет **объединять объекты в древовидные структуры и обрабатывать отдельные объекты и их композиции одинаково.**

- Основная цель: **представить группу объектов так, как если бы это был один объект;**
- Часто используется для **структурирования сложных иерархий объектов**, таких как графические компоненты или файловые системы.

Основные характеристики

- Существует **компонент (Component)** — общий интерфейс для всех объектов;
- **Лист (Leaf)** — объект, не имеющий дочерних элементов;
- **Составной объект (Composite)** — содержит дочерние компоненты и реализует методы компонента;
- Клиент **работает через интерфейс компонента**, не различая отдельные объекты и их композиции.

Применение

- Создание **иерархических структур**, например:
 - графические элементы GUI;
 - файловые и папочные системы;
 - организационные структуры;
- Когда нужно **обрабатывать объекты и их группы одинаково;**
- Упрощает **рекурсивную обработку и обход деревьев объектов.**

Преимущества

- Позволяет **работать с отдельными объектами и их группами единообразно;**
- Облегчает **расширение структуры объектов;**
- Упрощает **поддержку и модульность кода;**
- Повышает **гибкость при работе с иерархическими структурами.**

76. Структурный паттерн «Декоратор».

Структурный паттерн «Декоратор» (Decorator)

Понятие

Декоратор — это структурный паттерн проектирования, который позволяет динамически добавлять новые обязанности объекту, не изменяя его исходный код.

- Основная цель: **расширять функциональность объектов гибко и без наследования;**
- Часто используется, когда **разнообразие комбинаций поведения невозможно реализовать через классическую иерархию наследования.**

Основные характеристики

- Существует **компонент (Component)** — интерфейс или абстрактный класс;
- **Конкретный компонент (ConcreteComponent)** — объект с базовым поведением;
- **Декоратор (Decorator)** — содержит ссылку на компонент и реализует его интерфейс;
- Клиент **работает с объектом через интерфейс компонента**, не отличая декорированный объект от обычного.

Применение

- Добавление **нового поведения объектам в рантайме;**
- Расширение функциональности **GUI-компонентов, потоков ввода/вывода;**
- Используется для **логирования, проверки доступа, кэширования и других аспектов;**
- Позволяет создавать **комбинации обязанностей без дублирования кода.**

Преимущества

- Динамическое расширение объектов **без изменения их исходного кода;**
- Снижает необходимость **множества подклассов для каждой комбинации поведения;**
- Поддерживает **гибкую композицию функциональности;**
- Упрощает **поддержку и масштабирование системы.**

77. Структурный паттерн «Фасад».

Структурный паттерн «Фасад» (Facade)

Понятие

Фасад — это структурный паттерн проектирования, который предоставляет **упрощённый интерфейс к сложной системе классов**, скрывая её внутреннюю сложность.

- Основная цель: **облегчить взаимодействие с подсистемой и снизить связность клиента с конкретными компонентами;**
- Позволяет **сосредоточить работу с несколькими объектами через единый интерфейс.**

Основные характеристики

- Существует **Фасад (Facade)** — класс, который предоставляет простой интерфейс для клиента;
- **Подсистема (Subsystem)** — набор классов, с которыми работает фасад;
- Клиент **работает только с фасадом**, не взаимодействуя напрямую с внутренними классами;
- Фасад **инкапсулирует сложную логику и упрощает использование системы.**

Применение

- Упрощение работы с **сложными библиотеками и API;**
- Организация **единых точек входа** для подсистем;
- Используется в **GUI, системах работы с базами данных, сетевыми протоколами;**
- Позволяет **изолировать изменения внутренней реализации**, не затрагивая клиентский код.

Преимущества

- Снижает **сложность взаимодействия с подсистемой;**
- Обеспечивает **слабую связность между клиентом и подсистемой;**
- Упрощает **поддержку и масштабирование системы;**
- Делает систему **более понятной и удобной для использования.**

78. Поведенческий паттерн «Цепочка обязанностей».

Поведенческий паттерн «Цепочка обязанностей» (Chain of Responsibility)

Понятие

Цепочка обязанностей — это поведенческий паттерн проектирования, который позволяет передавать запрос по цепочке объектов, пока кто-то из них не обработает этот запрос.

- Основная цель: **снизить зависимость отправителя запроса от получателя;**
- Позволяет **строить динамическую цепочку обработки** и гибко распределять ответственность.

Основные характеристики

- Существует **абстрактный обработчик (Handler)** с методом обработки запроса и ссылкой на следующий элемент цепочки;
- **Конкретные обработчики (ConcreteHandler)** реализуют обработку запроса и решают, передавать его дальше или завершить;
- Клиент **отправляет запрос первому обработчику**, не зная, кто его обработает;
- Поддерживает **динамическое формирование цепочки** и изменение порядка обработчиков.

Применение

- Обработка **событий и команд** в системах GUI;
- Системы **логирования и фильтрации запросов;**
- Организация **серийной обработки данных** или проверок;
- Когда необходимо **делегировать ответственность нескольким объектам** без жёсткой привязки.

Преимущества

- Снижает **связность между отправителем и получателями запроса;**
- Позволяет **гибко изменять порядок и состав обработчиков;**
- Упрощает **добавление новых обработчиков без изменения существующего кода;**
- Повышает **масштабируемость и модульность системы.**

79. Поведенческий паттерн «Итератор».

Поведенческий паттерн «Итератор» (Iterator)

Понятие

Итератор — это поведенческий паттерн проектирования, который обеспечивает последовательный доступ к элементам коллекции без раскрытия её внутренней структуры.

- Основная цель: **отделить алгоритм обхода коллекции от самой коллекции;**
- Позволяет **унифицировать способ перебора элементов разных коллекций.**

Основные характеристики

- Существует **Интерфейс Итератора (Iterator)** с методами `hasNext()` и `next()`;
- **Конкретный итератор (ConcreteIterator)** реализует интерфейс и хранит состояние текущей позиции;
- **Коллекция (Aggregate)** предоставляет метод для получения итератора;
- Клиент **работает через итератор**, не зная внутреннего представления коллекции.

Применение

- Обход списков, множеств, карт и других коллекций;
- Реализация универсальных алгоритмов перебора;
- Поддержка различных структур данных, не раскрывая детали их хранения;
- Часто используется в **Stream API**, коллекциях **Java** и пользовательских структурах данных.

Преимущества

- Снижает связность между клиентом и коллекцией;
- Позволяет перебирать элементы независимо от внутренней структуры;
- Упрощает повторное использование и поддержку алгоритмов обхода;
- Поддерживает разные способы итерации для одной коллекции.

80. Поведенческий паттерн «Снимок».

Снимок (Memento) — это поведенческий паттерн проектирования, который позволяет **сохранять и восстанавливать внутреннее состояние объекта**, не раскрывая его внутреннюю структуру другим объектам.

- Основная цель: **обеспечить сохранение состояния для отката или восстановления**, сохраняя инкапсуляцию;
- Используется для **undo/redo операций и сохранения версий объектов**.

Основные характеристики

- Существует **Originator (Создатель)** — объект, состояние которого нужно сохранить;
- **Memento (Снимок)** — объект, хранящий состояние Originator;
- **Caretaker (Хранитель)** — управляет сохранением и восстановлением снимков;
- Клиент **не имеет доступа к внутреннему состоянию Originator**, работая только с Caretaker.

Применение

- Реализация **undo/redo операций** в текстовых редакторах;
- Сохранение **версий объектов**;
- В системах, где необходимо **восстановление состояния без нарушения инкапсуляции**;
- Используется для **игровых сохранений и настройки конфигураций**.

Преимущества

- Сохраняет **инкапсуляцию внутреннего состояния объекта**;
- Позволяет **восстанавливать объекты к предыдущему состоянию**;
- Упрощает **реализацию undo/redo функционала**;
- Обеспечивает **гибкое управление состояниями**.

81. Поведенческий паттерн «Состояние».

Поведенческий паттерн «Состояние» (State)

Понятие

Состояние (State) — это поведенческий паттерн проектирования, который позволяет **изменять поведение объекта в зависимости от его внутреннего состояния**, словно объект меняет класс во время выполнения.

- Основная цель: **инкапсулировать состояния и логику переходов**, избегая громоздких условных операторов;
- Используется для объектов, которые **ведут себя по-разному в разных состояниях**.

Основные характеристики

- Существует **Контекст (Context)** — объект, поведение которого зависит от состояния;
- **Состояние (State)** — интерфейс, определяющий поведение для конкретного состояния;
- **Конкретные состояния (ConcreteState)** реализуют поведение для каждого состояния;
- Контекст делегирует выполнение действий **текущему состоянию**, меняя его при необходимости.

Применение

- Управление **состояниями UI-компонентов** (кнопки, формы, меню);
- В **системах платежей и заказов**, где объект проходит несколько этапов обработки;
- Игровые объекты с **различными режимами поведения**;
- Любые ситуации, где требуется **динамическое изменение поведения объекта в зависимости от состояния**.

Преимущества

- **Снижает сложность кода за счёт отказа от больших условных операторов**;
- **Инкапсулирует поведение для каждого состояния**;
- **Позволяет легко добавлять новые состояния**, не меняя существующий код;
- **Повышает гибкость, модульность и поддерживаемость системы**.

82. Поведенческий паттерн «Шаблонный метод».

Поведенческий паттерн «Шаблонный метод» (Template Method)

Понятие

Шаблонный метод — это поведенческий паттерн проектирования, который определяет скелет алгоритма в базовом классе, позволяя подклассам реализовать конкретные шаги алгоритма без изменения его структуры.

- Основная цель: **повторное использование кода и стандартизация алгоритмов**, при сохранении возможности настройки отдельных шагов;
- Позволяет **централизованно управлять последовательностью операций**, оставляя детали реализации подклассам.

Основные характеристики

- **Абстрактный класс (AbstractClass)** содержит **шаблонный метод**, определяющий порядок выполнения шагов;
- Некоторые шаги реализованы в базовом классе, а другие — **абстрактные**, которые реализуют подклассы;
- **Подклассы (ConcreteClass)** реализуют абстрактные методы, определяя конкретное поведение;
- Шаблонный метод **не меняется**, гарантируя единый порядок выполнения операций.

Применение

- Обработка **стандартных алгоритмов с вариациями**;
- Генерация **отчётов, обработка данных, построение документов**;
- В системах, где требуется **единая последовательность действий с различными деталями реализации**;
- Часто используется вместе с **фреймворками**, позволяя пользователю расширять поведение.

Преимущества

- Позволяет **повторно использовать код базового класса**;
- Обеспечивает **единый порядок выполнения алгоритма**;
- Позволяет **изменять детали поведения без изменения общей структуры**;
- Упрощает **поддержку, расширение и тестирование кода**.

83. Поведенческий паттерн «Команда».

Поведенческий паттерн «Команда» (Command)

Понятие

Команда (Command) — это поведенческий паттерн проектирования, который инкапсулирует запрос как объект, позволяя параметризовать объекты с операциями, ставить их в очередь, логировать и поддерживать отмену действий.

- Основная цель: **разделить отправителя запроса и его исполнителя**, обеспечивая гибкость и расширяемость системы;
- Позволяет **обрабатывать действия как объекты**, управлять их выполнением и хранением.

Основные характеристики

- **Команда (Command)** — интерфейс с методом `execute()`;
- **Конкретная команда (ConcreteCommand)** реализует интерфейс и вызывает методы получателя (Receiver);
- **Получатель (Receiver)** — объект, который выполняет конкретные действия;
- **Отправитель (Invoker)** хранит и инициирует команды, не зная деталей реализации;
- Поддерживается **очередь команд, отложенное выполнение и отмена действий**.

Применение

- Реализация **undo/redo функционала**;
- Отложенное или **периодическое выполнение задач**;
- Управление **GUI-событиями** (кнопки, меню, действия пользователя);
- В **сетевых или распределённых системах**, где команды могут храниться и выполняться асинхронно.

Преимущества

- Разделяет **отправителя и исполнителя запроса**;
- Позволяет **хранить и управлять действиями как объектами**;
- Обеспечивает **гибкость в управлении задачами, очередями и отменой операций**;
- Упрощает **масштабирование и расширение системы**.

84. Поведенческий паттерн «Посредник».

Поведенческий паттерн «Посредник» (Mediator)

Понятие

Посредник (Mediator) — это поведенческий паттерн проектирования, который инкапсулирует взаимодействие множества объектов, позволяя им общаться через посредника, а не напрямую друг с другом.

- Основная цель: **снизить связанность между объектами и упростить управление взаимодействием;**
- Позволяет **централизовать логику взаимодействия**, делая систему более управляемой.

Основные характеристики

- **Посредник (Mediator)** — интерфейс или класс, управляющий коммуникацией между объектами;
- **Конкретный посредник (ConcreteMediator)** реализует координацию действий и хранит ссылки на объекты;
- **Коллеги (Colleague)** — объекты, взаимодействующие через посредника;
- Клиенты **не общаются напрямую**, все запросы проходят через посредника.

Применение

- Управление **сложными взаимодействиями между GUI-компонентами** (например, формы, кнопки, панели);
- Централизация логики **обмена сообщениями или событиями** между объектами;
- В **чатах, брокерах событий и системах управления потоками данных;**
- Когда требуется **уменьшить количество связей между объектами** и повысить модульность.

Преимущества

- **Снижает связность между объектами;**
- **Централизует логику взаимодействия**, упрощая поддержку;
- **Упрощает добавление новых коллег без изменения существующих;**
- **Повышает масштабируемость и управляемость системы.**

85. Поведенческий паттерн «Наблюдатель».

Поведенческий паттерн «Наблюдатель» (Observer)

Понятие

Наблюдатель (Observer) — это поведенческий паттерн проектирования, который **определяет зависимость «один ко многим» между объектами**, чтобы при изменении состояния одного объекта **все зависимые объекты уведомлялись и автоматически обновлялись**.

- Основная цель: **обеспечить слабую связность между объектами**, при этом синхронизируя их состояния;
- Часто используется для **событийных систем и обмена данными между компонентами**.

Основные характеристики

- **Субъект (Subject)** — объект, состояние которого отслеживается;
- **Наблюдатели (Observer)** — объекты, которые получают уведомления об изменениях состояния субъекта;
- Субъект хранит **список наблюдателей** и уведомляет их при изменении состояния;
- Наблюдатели **обновляют своё состояние или выполняют действия** в ответ на уведомление.

Применение

- Системы **GUI и событийные интерфейсы** (кнопки, формы, панели);
- Подписка на **изменения данных и состояния объектов**;
- Реализация **событийных механизмов, уведомлений и реактивного программирования**;
- Используется в **MVC и MVVM архитектурах** для синхронизации модели и представления.

Преимущества

- Обеспечивает **слабую связность между объектами**;
- Позволяет **добавлять новых наблюдателей без изменения субъекта**;
- Облегчает **масштабирование и повторное использование компонентов**;
- Упрощает **поддержку и расширение системы**.

86. Поведенческий паттерн «Стратегия».

Поведенческий паттерн «Стратегия» (Strategy)

Понятие

Стратегия (Strategy) — это поведенческий паттерн проектирования, который определяет семейство алгоритмов, инкапсулирует их и делает взаимозаменяемыми, позволяя выбирать алгоритм во время выполнения.

- Основная цель: **разделить использование алгоритма и его реализацию**, чтобы упростить изменение поведения объекта;
- Используется, когда **разные способы выполнения задачи должны быть взаимозаменяемыми**.

Основные характеристики

- **Контекст (Context)** — объект, который использует стратегию;
- **Интерфейс стратегии (Strategy)** — общий интерфейс для всех алгоритмов;
- **Конкретные стратегии (ConcreteStrategy)** — реализация различных алгоритмов;
- Контекст делегирует выполнение задачи выбранной стратегии, меняя её при необходимости.

Применение

- Выбор разных алгоритмов сортировки, фильтрации или расчётов;
- В системах с динамическим поведением, когда алгоритм может меняться во время работы;
- Управление вариантами бизнес-логики или правил обработки данных;
- Используется в GUI, системах маршрутизации и игровых механиках.

Преимущества

- Позволяет легко менять алгоритмы во время выполнения;
- Снижает связность между контекстом и конкретными реализациями;
- Упрощает добавление новых алгоритмов без изменения существующего кода;
- Повышает гибкость, повторное использование и поддерживаемость системы.

87. Поведенческий паттерн «Посетитель».

Поведенческий паттерн «Посетитель» (Visitor)

Понятие

Посетитель (Visitor) — это поведенческий паттерн проектирования, который разделяет алгоритмы и структуру объектов, позволяя добавлять новые операции для объектов без изменения их классов.

- Основная цель: расширять функциональность объектов, сохраняя инкапсуляцию их структуры;
- Полезен, когда необходимо выполнять разнообразные операции над объектами сложной структуры.

Основные характеристики

- **Элемент (Element)** — интерфейс или абстрактный класс объекта, который может быть «посещён»;
- **Конкретные элементы (ConcreteElement)** — реализуют метод `accept(Visitor)` для принятия посетителя;
- **Посетитель (Visitor)** — интерфейс, определяющий операции для разных типов элементов;
- **Конкретный посетитель (ConcreteVisitor)** — реализует операции для каждого типа элемента;
- Элемент делегирует выполнение операции посетителю, не меняя свою структуру.

Применение

- Выполнение операций над объектами сложной структуры (например, AST, графы, DOM);
- Разделение алгоритмов и объектов данных;
- Реализация различных отчётов, обработок и обходов структур;
- Используется в компиляторах, системах анализа данных и документации.

Преимущества

- Позволяет добавлять новые операции без изменения классов элементов;
- Сохраняет инкапсуляцию и структуру объектов;
- Упрощает расширение функциональности и поддержку кода;
- Удобен для обхода сложных иерархий объектов.

88. Принципы SOLID.

Принципы SOLID

SOLID — это набор пяти базовых принципов объектно-ориентированного проектирования, направленных на **повышение гибкости, расширяемости и сопровождаемости кода**.

1. S — Single Responsibility Principle (Принцип единственной ответственности)

- Каждый класс должен иметь **только одну причину для изменения**, то есть выполнять **одну чётко определённую задачу**.
- Это **упрощает поддержку и тестирование кода**.

2. O — Open/Closed Principle (Принцип открытости/закрытости)

- Классы должны быть **открыты для расширения, но закрыты для изменения**.
- Новую функциональность добавляем через **наследование или композицию**, не меняя существующий код.

3. L — Liskov Substitution Principle (Принцип подстановки Лисков)

- Объекты подкласса должны **заменять объекты базового класса без нарушения работы программы**.
- Подклассы должны **соблюдать контракт родительского класса**, чтобы код был предсказуемым.

4. I — Interface Segregation Principle (Принцип разделения интерфейсов)

- Клиенты не должны зависеть от интерфейсов, которые они **не используют**.
- Лучше иметь **несколько узкоспециализированных интерфейсов**, чем один большой универсальный.

5. D — Dependency Inversion Principle (Принцип инверсии зависимостей)

- Модули верхнего уровня не должны зависеть от модулей нижнего уровня.
- **Оба типа должны зависеть от абстракций**, а не от конкретных реализаций.
- Это **повышает гибкость, повторное использование и тестируемость кода**.